4-11-2026

DOCKER

PROYECTO FINAL

Tatiana Pérez Tamayo

INSTRUCTOR
Aristedes Novoa Arbildo

Contenido

INTRODUCCIÓN 2
DOCKER CHEAT SEET 3
CONTENEDORES SIMPLES Y VOLUMEN MONGODB 4
REDES Y CONECTIVIDAD 13
CONECTANDO TRES CONTENEDORES 14
REGISTRO DE IMÁGENES DOCKER HUB 19
DOCKER COMPOSE 20
DOCKER COMPOSE CON BUILD 29
SECRETS Y VOLUMEN PERSISTENTE 30
OTROS COMANDO DE INVESTIGACIÓN 31

INTRODUCCIÓN

Este entregable documenta el desarrollo y la contenerización de una aplicación de inventario en Python. La solución incluye una API construida con FastAPI, una base de datos MongoDB, una interfaz visual, imágenes Docker propias, ejecución de contenedores, volúmenes para persistencia, redes internas, publicación en Docker Hub y un stack con Docker Compose. Las evidencias muestran cómo se prueba la aplicación localmente y cómo sus servicios se comunican de forma controlada mediante los puertos, la red y los secretos configurados.

DOCKER CHEAT SEET

DOCKER CHEAT SEET

COMANDOS GENERALES

- docker version | Ver la versión del cliente y del motor Docker.
- docker info | Mostrar información detallada del motor, recursos y configuración.
- docker --help | Mostrar ayuda general de Docker.
- docker system df | Ver el espacio ocupado por recursos Docker.

CONTENEDORES

- docker run --name <nombre_del_contenedor> <nombre_de_la_imagen> | Crear y ejecutar un contenedor a partir de una imagen, con un nombre personalizado.
- docker run -p <puerto_del_host>:<puerto_del_contenedor> <nombre_de_la_imagen> | Ejecutar un contenedor y publicar los puertos del contenedor en el host.
- docker run -d <nombre_de_la_imagen> | Ejecutar un contenedor en segundo plano
- docker start|stop <nombre_del_contenedor> (o <id_del_contenedor>) | Iniciar o detener un contenedor existente.
- docker rm <nombre_del_contenedor> | Eliminar un contenedor detenido.
- docker exec -it <nombre_del_contenedor> sh | Abrir un shell dentro de un contenedor en ejecución.
- docker logs -f <nombre_del_contenedor> | Obtener y seguir los registros de un contenedor.
- docker inspect <nombre_del_contenedor> (o <id_del_contenedor>) | Para inspeccionar un contenedor en ejecución.
- docker ps | Para mostrar una lista de los contenedores que se están ejecutando actualmente.
- docker ps --all | Mostrar una lista de todos los contenedores de Docker (en ejecución y detenidos).
- docker container stats | Ver estadísticas de uso de recursos.

IMAGENES

- docker build -t . | Crear una imagen a partir de un Dockerfile.
- docker build -t <nombre_de_la_imagen> . –no-cache | Crear una imagen a partir de un Dockerfile sin utilizar la caché.
- docker images | Mostrar las imágenes locales.
- docker rmi <nombre_de_la_imagen> | Eliminar una imagen.
- docker image prune | Eliminar todas las imágenes que no se utilizan.

REDES

- docker network ls | Listar redes disponibles.
- docker network create <nombre_de_la_red> | Crear una red interna personalizada.
- docker network connect <nombre_de_la_red> <nombre_del_contenedor> | Conectar un contenedor existente a una red.
- docker network disconnect <nombre_de_la_red> <nombre_del_contenedor> | Desconectar un contenedor de una red.
- docker network inspect <nombre_de_la_red> | Ver los contenedores y la configuración de una red.
- docker network prune | Eliminar redes no utilizadas

VOLUMENES

- docker volume create <nombre_del_volumen> | Crear un volumen administrado por Docker.
- docker volume ls | Listar volúmenes disponibles.
- docker run -d --name <nombre_del_contenedor> -v <nombre_del_volumen>:/data/db mongo:8.0 | Crear MongoDB y montar el volumen para conservar sus datos.
- docker volume rm <nombre_del_volumen> | Eliminar el volumen.

Figura 1. *Hoja de ayuda personalizada con comandos Docker utilizados durante el proyecto.*

INVENTARIO PYTHON Y DOCKER

Proyecto de una API Python con FastAPI, base de datos MongoDB y una interfaz visual sencilla.

CONTENEDORES SIMPLES Y VOLUMEN MONGODB

- Creacion de Volumen

docker volume create mongo-CURS-000662-data

```
PowerShell 7.6.9
PS C:\Users\User> docker volume create mongo-CURS-000662-data
mongo-CURS-000662-data
PS C:\Users\User>
```

Figura 2. *Creación del volumen mongo-CURS-000662-data para conservar los datos de MongoDB.*

- Creacion de contenedor de MongoDB

```
docker run --name mongo-CURS-000662-Docker -d `
 -e MONGO_INITDB_ROOT_USERNAME=root `
 -e MONGO_INITDB_ROOT_PASSWORD=123456 `
 -p 27017:27017 `
 -v mongo-CURS-000662-data:/data/db `
 mongo:8.0.34-noble
```

```
PS C:\Users\User> docker run --name mongo-CURS-000662-Docker -d `
>>   -e MONGO_INITDB_ROOT_USERNAME=root `
>>   -e MONGO_INITDB_ROOT_PASSWORD=123456 `
>>   -p 27017:27017 `
>>   -v mongo-CURS-000662-data:/data/db `
>>   mongo:8.0.34-noble
Unable to find image 'mongo:8.0.34-noble' locally
8.0.34-noble: Pulling from library/mongo
c65eb1fed990: Pull complete
5332cf0f0742: Pull complete
1170af5a3d82: Pull complete
d1d7ab01ea82: Pull complete
cda377d20aef: Pull complete
7a48b8f0e5c7: Pull complete
493752c97794: Pull complete
0934021729a2: Pull complete
0b2a3b5d1b31: Download complete
878968fa0b72: Download complete
Digest: sha256:3c64937121b9437095921c4bb2569911084c05f7b88e4a263d62a8959d2b2399
Status: Downloaded newer image for mongo:8.0.34-noble
aeaad35923518405bd1f216f4d80fef3b223b4d3b5b3bf20b81d669a38380132
PS C:\Users\User>
```

Figura 3. *Creación del contenedor MongoDB con puerto y volumen configurados.*

- Se comprueba estado del contenedor

```
PS C:\Users\User> docker ps
CONTAINER ID   IMAGE                              COMMAND                  CREATED          STATUS          PORTS                                                  NAMES
aeaad3592351   mongo:8.0.34-noble                 "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes    0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp        mongo-CURS-000662-Docker
42267c8b0247   ttamayo08/api-employees:1.0.0      "java -jar /api-empl…"   4 hours ago      Up 4 hours      0.0.0.0:8082->8081/tcp, [::]:8082->8081/tcp            api-employees-v2
dcdc663f2f8a   ttamayo08/api-employees:1.0.0      "java -jar /api-empl…"   4 hours ago      Up 4 hours      0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp            api-employees
e5c87e155032   mysql:9.0                          "docker-entrypoint.s…"   4 hours ago      Up 4 hours      3306/tcp, 33060/tcp                                    mysql-CURS-000662-Docker-dev
PS C:\Users\User>
```

Figura 4. *Verificación de que el contenedor MongoDB se encuentra en ejecución mediante docker ps.*

```
PS C:\Users\User> docker logs mongo-CURS-000662-Docker
about to fork child process, waiting until server is ready for connections.
forked process: 28

{"t":{"$date":"2026-10-30T03:05:10.017+00:00"},"s":"I",  "c":"CONTROL",  "id":20698,   "ctx":"main","msg":"***** SERVER RESTARTED ***
**"}
{"t":{"$date":"2026-10-30T03:05:10.033+00:00"},"s":"I",  "c":"EXECUTOR", "id":11280000,"ctx":"main","msg":"Starting thread pool","att
r":{"poolName":"parallel execution pool","numThreads":0,"minThreads":0,"maxThreads":128}}
{"t":{"$date":"2026-10-30T03:05:10.036+00:00"},"s":"I",  "c":"CONTROL",  "id":23285,   "ctx":"main","msg":"Automatically disabling TL
S 1.0, to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'"}
{"t":{"$date":"2026-10-30T03:05:10.036+00:00"},"s":"I",  "c":"CONTROL",  "id":5945603, "ctx":"main","msg":"Multi threading initialize
d"}
{"t":{"$date":"2026-10-30T03:05:10.037+00:00"},"s":"I",  "c":"NETWORK",  "id":4648601, "ctx":"main","msg":"Implicit TCP FastOpen unav
ailable. If TCP FastOpen is required, set at least one of the related parameters","attr":{"relatedParameters":["tcpFastOpenServer","t
cpFastOpenClient","tcpFastOpenQueueSize"]}}
{"t":{"$date":"2026-10-30T03:05:10.038+00:00"},"s":"I",  "c":"EXECUTOR", "id":11280000,"ctx":"main","msg":"Starting thread pool","att
r":{"poolName":"AuthorizationManager","numThreads":0,"minThreads":0,"maxThreads":1000000000}}
{"t":{"$date":"2026-10-30T03:05:10.038+00:00"},"s":"I",  "c":"EXECUTOR", "id":11280000,"ctx":"main","msg":"Starting thread pool","att
r":{"poolName":"OCSPCache","numThreads":0,"minThreads":0,"maxThreads":8}}
{"t":{"$date":"2026-10-30T03:05:10.038+00:00"},"s":"I",  "c":"EXECUTOR", "id":11280000,"ctx":"main","msg":"Starting thread pool","att
r":{"poolName":"ThreadPool1","numThreads":0,"minThreads":0,"maxThreads":1}}
{"t":{"$date":"2026-10-30T03:05:10.038+00:00"},"s":"I",  "c":"NETWORK",  "id":4915701, "ctx":"main","msg":"Initialized wire specifica
tion","attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"maxWireVersion":25},"incomingInternalClient":{"minWireVersion":0,
"maxWireVersion":25},"outgoing":{"minWireVersion":6,"maxWireVersion":25},"isInternalClient":true}}}
{"t":{"$date":"2026-10-30T03:05:10.039+00:00"},"s":"I",  "c":"EXECUTOR", "id":11280000,"ctx":"main","msg":"Starting thread pool","att
r":{"poolName":"ThreadPool2","numThreads":0,"minThreads":0,"maxThreads":1}}
```

Figura 5. *Consulta de los registros de MongoDB para confirmar su inicio correcto.*

- Preparación del entorno para ejecutar el proyecto

Se crea un entorno virtual llamado .venv, que funciona como un espacio aislado para las librerías del proyecto. Luego se activa ese entorno en PowerShell para que los comandos python y pip trabajen dentro de él. Finalmente, se instalan las dependencias definidas en requirements.txt, como FastAPI para la API, PyMongo para conectarse con MongoDB, Uvicorn para ejecutar el servidor y Jinja2 para mostrar la interfaz

py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt

```
PS C:\Users\User> cd C:\Users\User\Projects\Proyecto_Docker
PS C:\Users\User\Projects\Proyecto_Docker> py -3.12 -m venv .venv
PS C:\Users\User\Projects\Proyecto_Docker> .\.venv\Scripts\Activate.ps1
```

Figura 6. *Creación del entorno virtual de Python para aislar las dependencias del proyecto.*



Figura 7. *Activación del entorno virtual .venv en PowerShell.*

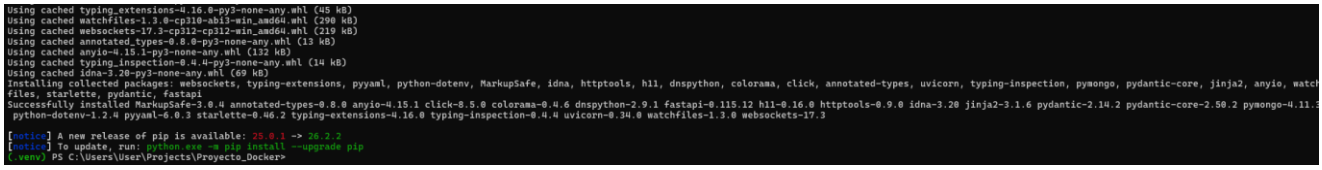


Figura 8. *Instalación de las librerías requeridas por la API desde requirements.txt.*

Se configuran las variables de entorno que indican la conexión con MongoDB, el nombre de la base de datos y la versión de la aplicación; luego se inicia FastAPI con Uvicorn en el puerto 8091. De esta forma, la API queda disponible localmente para ser probada desde el navegador, Postman o la interfaz visual del proyecto.

```
(.venv) PS C:\Users\User\Projects\Proyecto_Docker> $env:MONGO_URL='mongodb://root:123456@localhost:27017/?authSource=admin'
(.venv) PS C:\Users\User\Projects\Proyecto_Docker> $env:MONGO_DATABASE='inventario_curso'
(.venv) PS C:\Users\User\Projects\Proyecto_Docker> $env:APP_VERSION='local'
(.venv) PS C:\Users\User\Projects\Proyecto_Docker>
(.venv) PS C:\Users\User\Projects\Proyecto_Docker> uvicorn app.main:app --reload --port 8091
INFO:     Will watch for changes in these directories: ['C:\\Users\\User\\Projects\\Proyecto_Docker']
INFO:     Uvicorn running on http://127.0.0.1:8091 (Press CTRL+C to quit)
INFO:     Started reloader process [38188] using WatchFiles
INFO:     Started server process [11908]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:59585 - "GET /api/products HTTP/1.1" 200 OK
INFO:     127.0.0.1:59585 - "GET /api/products HTTP/1.1" 200 OK
INFO:     127.0.0.1:59480 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:61575 - "GET /api/products HTTP/1.1" 200 OK
INFO:     127.0.0.1:61578 - "GET /health HTTP/1.1" 200 OK
INFO:     127.0.0.1:60388 - "GET /api/products HTTP/1.1" 200 OK
INFO:     127.0.0.1:62787 - "POST /api/products HTTP/1.1" 201 Created
INFO:     127.0.0.1:64948 - "GET /api/products HTTP/1.1" 200 OK
INFO:     127.0.0.1:52174 - "POST /api/products HTTP/1.1" 201 Created
INFO:     127.0.0.1:55238 - "GET /api/products HTTP/1.1" 200 OK
```

Figura 9. *Configuración de las variables de entorno y arranque local de la API con Uvicorn.*

http://localhost:8091/health

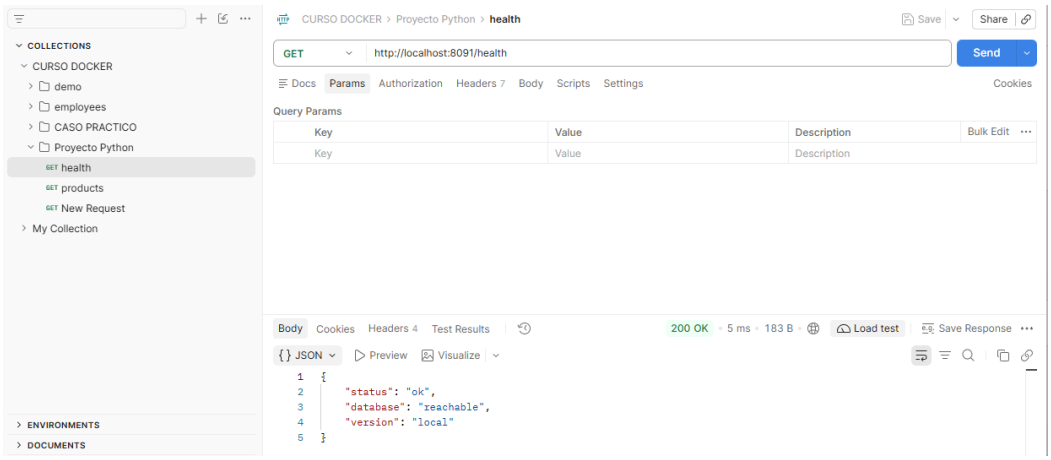


Figura 10. *Prueba del endpoint health para confirmar que la API está disponible.*

http://localhost:8091/api/products

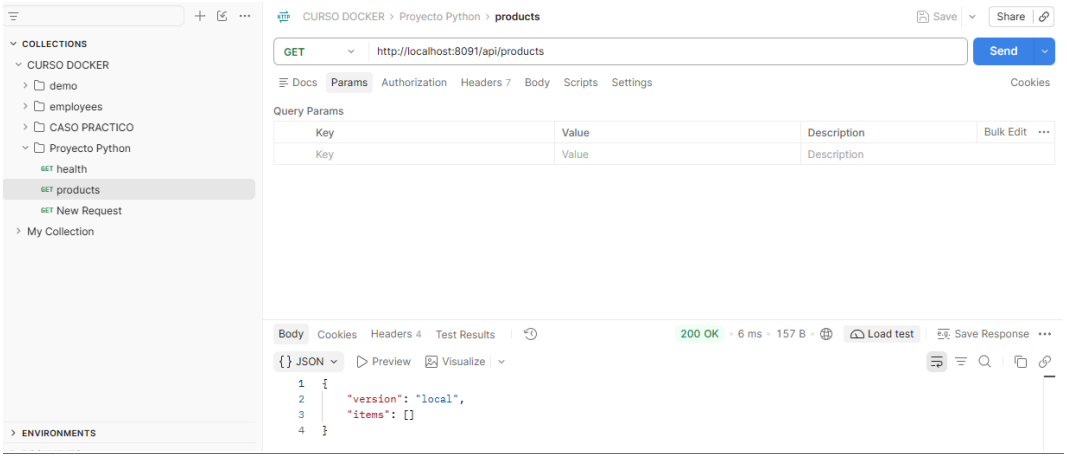


Figura 11. *Consulta inicial de productos mediante el endpoint api products.*

POST http://localhost:8091/api/products

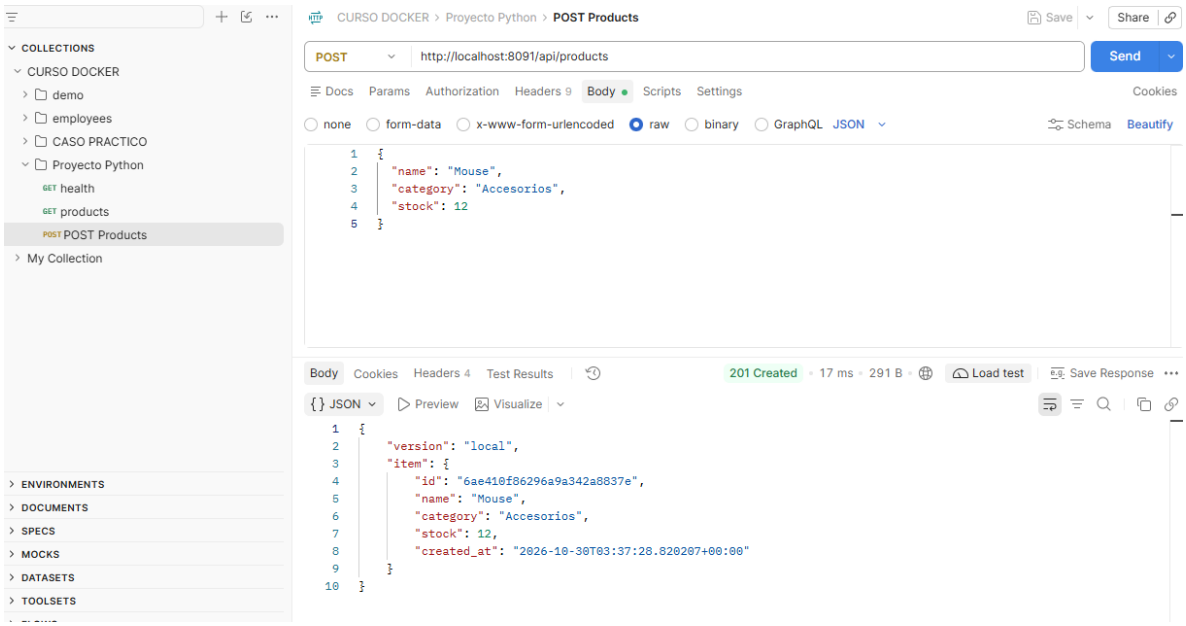


Figura 12. *Creación de un producto por medio de una solicitud POST a la API.*

http://localhost:8091/api/products - Con productos agregados

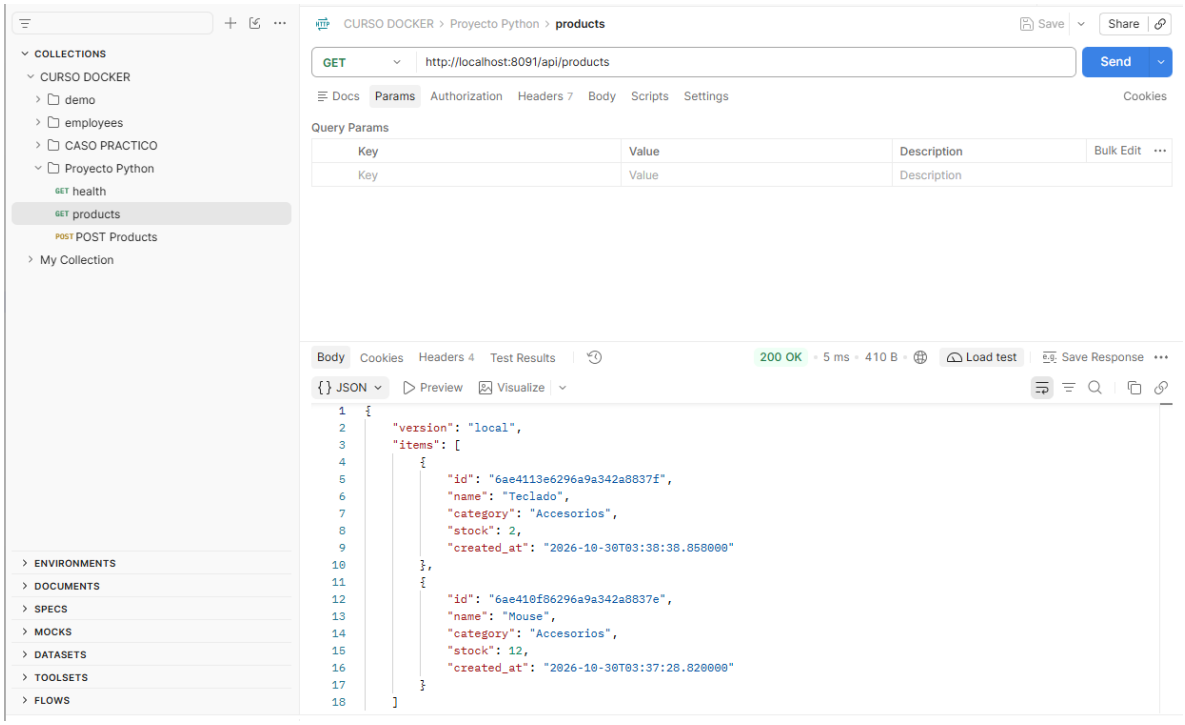


Figura 13. *Consulta de productos después de insertar información en MongoDB.*

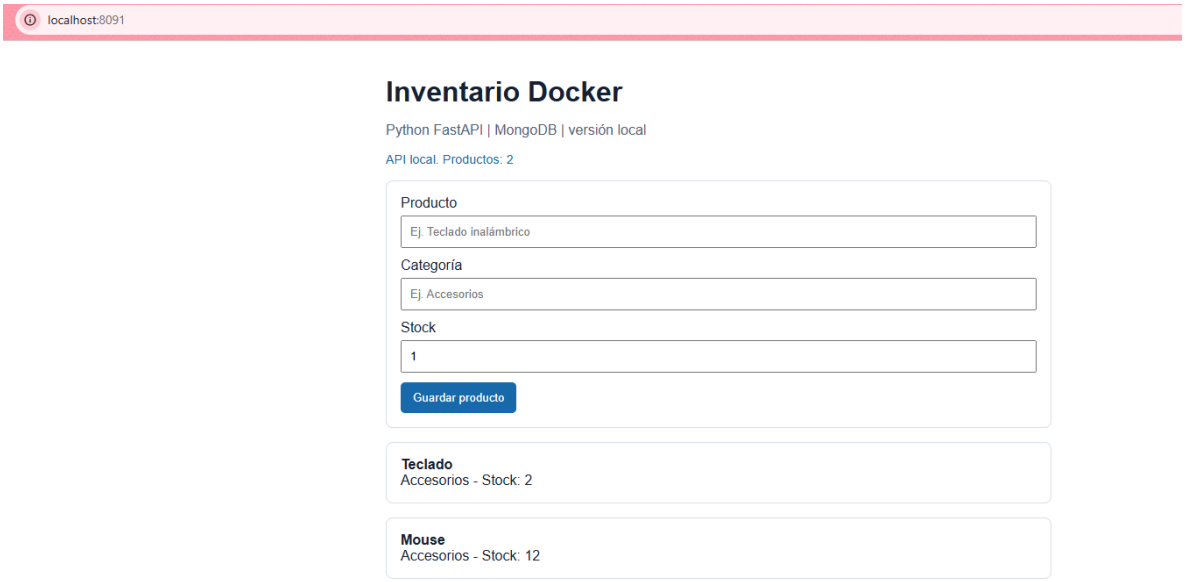


Figura 14. *Visualización de los productos creados desde la interfaz de la aplicación.*

- Se ingresa al contenedor de la DB y se logra ver la información ingresada.

docker exec -it mongo-CURS-000662-Docker mongosh -u root -p 123456
use inventario_curso
db.products.find()

```
PS C:\Users\User> docker exec -it mongo-CURS-000662-Docker mongosh -u root -p 123456
Current Mongosh Log ID: 6ae412d5056bb721b3411bfa
Connecting to:          mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          8.0.34
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only, and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-10-30T03:05:12.544+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
   2026-10-30T03:05:13.020+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2026-10-30T03:05:13.020+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2026-10-30T03:05:13.020+00:00: We suggest setting the contents of sysfsFile to 0.
   2026-10-30T03:05:13.021+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
------

test> use inventario_curso
| db.products.find()
|
switched to db inventario_curso
inventario_curso> db.products.find()
[
  {
    _id: ObjectId('6ae410f86296a9a342a8837e'),
    name: 'Mouse',
    category: 'Accesorios',
    stock: 12,
    created_at: ISODate('2026-10-30T03:37:28.820Z')
  },
  {
    _id: ObjectId('6ae4113e6296a9a342a8837f'),
    name: 'Teclado',
    category: 'Accesorios',
    stock: 2,
    created_at: ISODate('2026-10-30T03:38:38.858Z')
  }
]
inventario_curso>
```

Se detiene el API local.

Figura 15. *Detención de la API ejecutada localmente antes de usar su contenedor.*

- Se procede a crear el dockerfile

FROM	Parte de una imagen con Python instalado.
LABEL	Agrega metadatos descriptivos a la imagen Docker.
WORKDIR	Define /app como carpeta de trabajo interna.
COPY requirements.txt	Copia la lista de dependencias.
RUN pip install	Instala FastAPI, Uvicorn, PyMongo y Jinja2.
COPY app	Copia código y UI dentro de la imagen.
EXPOSE 8091	Documenta el puerto donde escucha la API.
CMD	Inicia Uvicorn cuando el contenedor arranca.

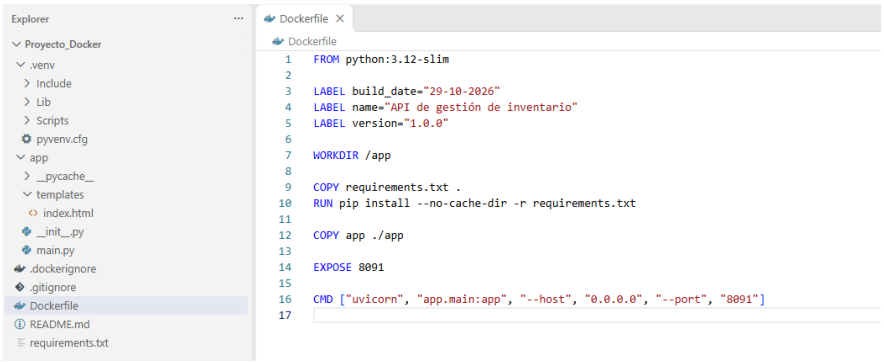


Figura 16. *Definición del comando CMD en el Dockerfile para iniciar Uvicorn automáticamente.*

- Creación de imagen

docker build -t ttamayo08/inventario-api:1.0.0 .
docker image ls inventario-api

```
(.venv) PS C:\Users\User\Projects\Proyecto_Docker> docker build -t ttamayo08/inventario-api:1.0.0 .
[+] Building 45.8s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 348B
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 91B
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => => sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3 12.12MB / 12.12MB
 => => sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35 1.29MB / 1.29MB
 => => sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e 251B / 251B
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5 29.86MB / 29.86MB
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5
 => => extracting sha256:3eb77115150676568f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35
 => => extracting sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3
 => => extracting sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e
 => [internal] load build context
 => => transferring context: 10.77kB
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY app ./app
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:6ed27092b270797a8221768e3aae6ebee322f9d5c9886276a8faddb1215419a1
 => => exporting config sha256:5633023d217b5617b9760b31ad743b068c5b72aa97f9181d4355e1ceeab98dc6
 => => exporting attestation manifest sha256:d2027c9b2edd28baf5fb064e044508fe2d544db2e64d32e68a8677e541993373
 => => exporting manifest list sha256:076243afc56db7fba72b803751fd8929837e3b590f0021dcf2b5f727519b7b7d
 => => naming to docker.io/ttamayo08/inventario-api:1.0.0
 => => unpacking to docker.io/ttamayo08/inventario-api:1.0.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/vsycr45gmyvqnvnq7nxmvo9v5
```

Figura 17. *Construcción de la imagen propia de la API de inventario.*

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/vsycr45gmyvqnvnq7nxmvo9v5
(.venv) PS C:\Users\User\Projects\Proyecto_Docker> docker image ls

IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
mongo:8.0.34-noble                   3c64937121b9       1.28GB          319MB    U
mysql:9.0                            92dc86967801        803MB          171MB    U
ttamayo08/api-employees:1.0.0        5e092f0dc335        781MB          281MB    U
ttamayo08/inventario-api:1.0.0       076243afc56d        264MB         62.8MB
(.venv) PS C:\Users\User\Projects\Proyecto_Docker>
```

Figura 18. *Verificación de la imagen generada mediante docker image ls.*

- Creación de contenedor API

```
docker run --name inventario-api-local -d `
 -p 8091:8091 `
 -e MONGO_URL='mongodb://root:123456@host.docker.internal:27017/?authSource=admin' `
 -e MONGO_DATABASE=inventario_curso `
 -e APP_VERSION=v1 `
 ttamayo08/inventario-api:1.0.0
```

```
PS C:\Users\User\Projects\Proyecto_Docker> docker run --name inventario-api-local -d `
>>   -p 8091:8091 `
>>   -e MONGO_URL='mongodb://root:123456@host.docker.internal:27017/?authSource=admin' `
>>   -e MONGO_DATABASE=inventario_curso `
>>   -e APP_VERSION=v1 `
>>   ttamayo08/inventario-api:1.0.0
742deead9ba6cc0c85f2d8071b3d565eb8729604c95c7919b66f2df5f235b26a
PS C:\Users\User\Projects\Proyecto_Docker> docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED              STATUS              PORTS                                                  NAMES
742deead9ba6   ttamayo08/inventario-api:1.0.0   "uvicorn app.main:ap…"   About a minute ago   Up About a minute   0.0.0.0:8091->8091/tcp, [::]:8091->8091/tcp            inventario-api-local
aeaad3592351   mongo:8.0.34-noble             "docker-entrypoint.s…"   About an hour ago    Up About an hour    0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp        mongo-CURS-000662-Docker
42267c8b0247   ttamayo08/api-employees:1.0.0    "java -jar /api-empl…"   5 hours ago          Up 5 hours          0.0.0.0:8082->8081/tcp, [::]:8082->8081/tcp            api-employees-v2
dcdc663f2f8a   ttamayo08/api-employees:1.0.0    "java -jar /api-empl…"   5 hours ago          Up 5 hours          0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp            api-employees
e5c87e159032   mysql:9.0                      "docker-entrypoint.s…"   5 hours ago          Up 5 hours          3306/tcp, 33060/tcp                                    mysql-CURS-000662-Docker-dev
PS C:\Users\User\Projects\Proyecto_Docker>
```

Figura 19. *Ejecución del contenedor de la API con el puerto 8091 publicado.*

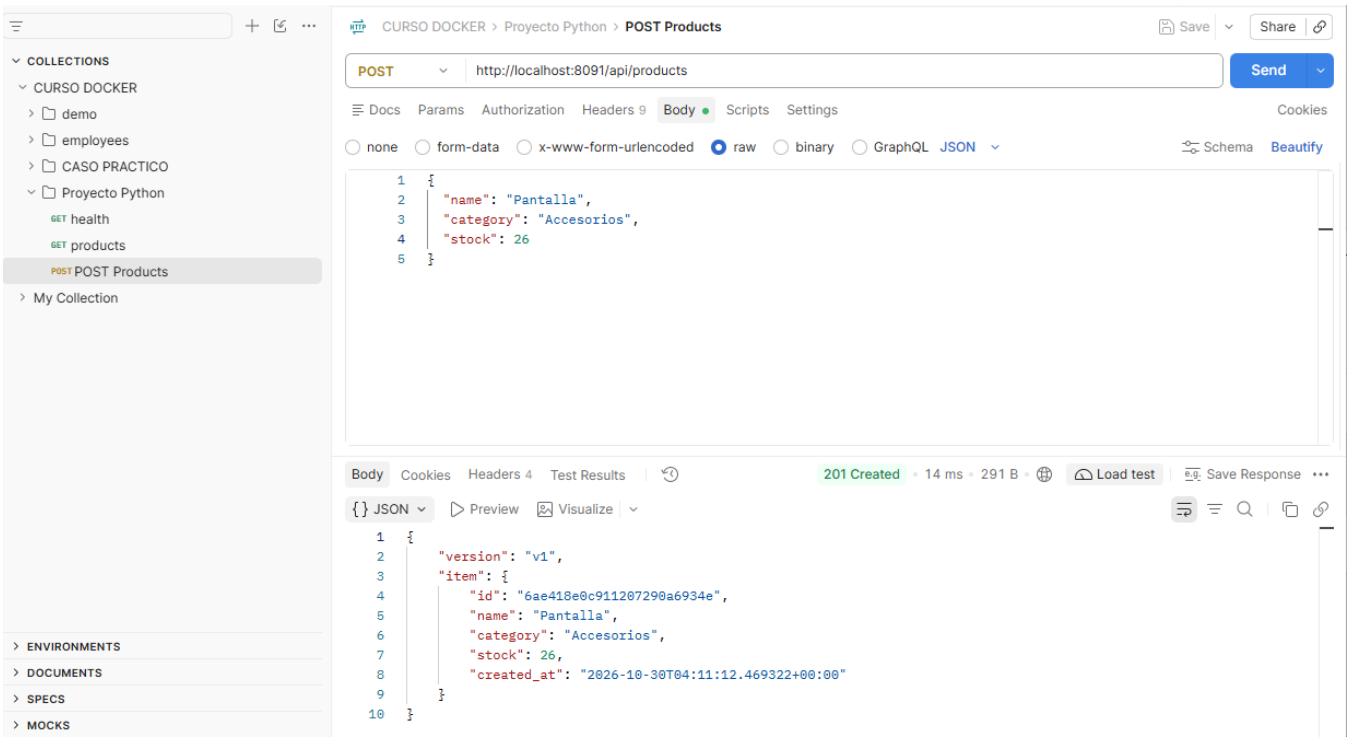


Figura 20. *Configuración de la conexión desde el contenedor API hacia MongoDB.*

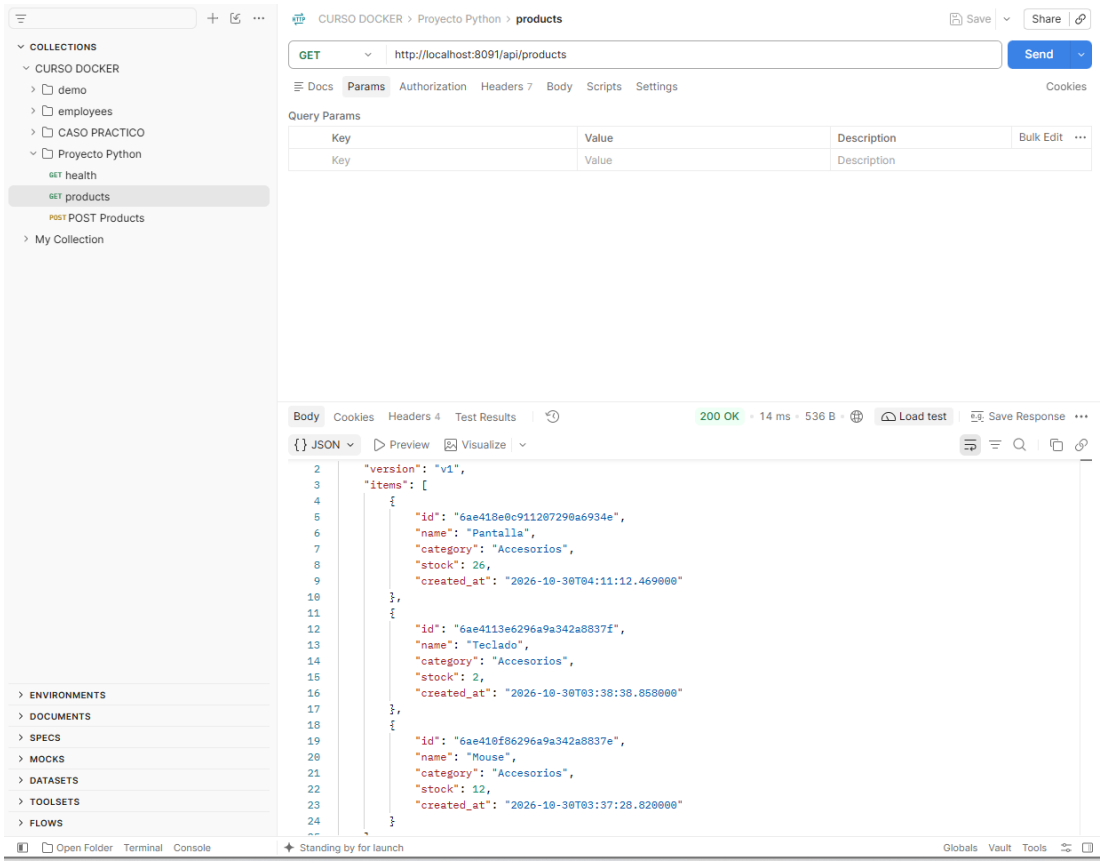


Figura 21. *Confirmación de que el contenedor de la API está activo.*

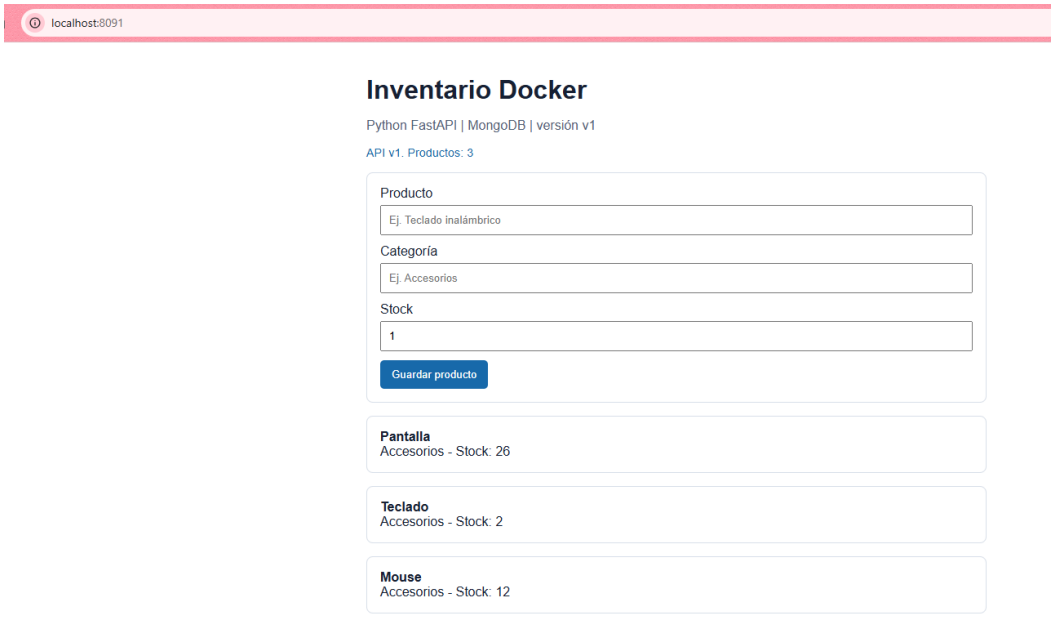


Figura 22. *Prueba de la API ejecutada desde su imagen Docker.*

```
PS C:\Users\User\Projects\Proyecto_Docker> docker logs inventario-api-local
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8091 (Press CTRL+C to quit)
INFO:     172.17.0.1:48214 - "GET / HTTP/1.1" 200 OK
INFO:     172.17.0.1:48214 - "GET /api/products HTTP/1.1" 200 OK
INFO:     172.17.0.1:48214 - "GET / HTTP/1.1" 200 OK
INFO:     172.17.0.1:48214 - "GET /api/products HTTP/1.1" 200 OK
INFO:     172.17.0.1:33958 - "GET /api/products HTTP/1.1" 200 OK
INFO:     172.17.0.1:33358 - "POST /api/products HTTP/1.1" 201 Created
INFO:     172.17.0.1:43746 - "GET /api/products HTTP/1.1" 200 OK
INFO:     172.17.0.1:36584 - "GET / HTTP/1.1" 200 OK
INFO:     172.17.0.1:36584 - "GET /api/products HTTP/1.1" 200 OK
PS C:\Users\User\Projects\Proyecto_Docker>
```

Figura 23. *Acceso a la interfaz visual publicada por el contenedor de la API.*

- Se eliminan los dos contenedores creados.

docker rm -f inventario-api-local mongo-CURS-000662-Docker

```
PS C:\Users\User\Projects\Proyecto_Docker> docker rm -f inventario-api-local mongo-CURS-000662-Docker
inventario-api-local
mongo-CURS-000662-Docker
PS C:\Users\User\Projects\Proyecto_Docker>
```

Figura 24. *Eliminación de los contenedores iniciales para preparar la comunicación por red interna.*

REDES Y CONECTIVIDAD

- Se crea la red inventory_network

docker network create inventory_network

```
PS C:\Users\User\Projects\Proyecto_Docker> docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
4ebddc3486de   bridge            bridge    local
0f39901e44c7   employee_network  bridge    local
4cd96a4b2b68   host              host      local
d288e885a060   none              null      local
PS C:\Users\User\Projects\Proyecto_Docker> docker network create inventory_network
7fbce75c3e6f7d14f4f4f8899401997b65d0701a1f99bb198477e8c4d0f47a27
PS C:\Users\User\Projects\Proyecto_Docker> docker network ls
NETWORK ID     NAME               DRIVER    SCOPE
4ebddc3486de   bridge             bridge    local
0f39901e44c7   employee_network   bridge    local
4cd96a4b2b68   host               host      local
7fbce75c3e6f   inventory_network  bridge    local
d288e885a060   none               null      local
PS C:\Users\User\Projects\Proyecto_Docker>
```

Figura 25. *Creación de la red personalizada inventory_network.*

CONECTANDO TRES CONTENEDORES

Se crea el contenedor de MongoDB pero sin el puerto y conectado a la nueva red inventory_network

```
PS C:\Users\User\Projects\Proyecto_Docker> docker run --name mongo-CURS-000662-Docker -d `
>>   -e MONGO_INITDB_ROOT_USERNAME=root `
>>   -e MONGO_INITDB_ROOT_PASSWORD=123456 `
>>   -v mongo-CURS-000662-data:/data/db `
>>   --network inventory_network `
>>   mongo:8.0.34-noble
92c5951476a691ce926899b70a9b59d7f236fd5457b1ebfe0944ba3603044b71
```

Figura 26. *Creación del contenedor MongoDB conectado a la red interna sin publicar su puerto.*

Se crea el contenedor de la API con la conexión directamente al contenedor de la DB y a la nueva red inventory_network

```
PS C:\Users\User\Projects\Proyecto_Docker> docker run --name inventario-api-v1 -d `
>>   -p 8091:8091 `
>>   -e MONGO_URL='mongodb://root:123456@mongo-CURS-000662-Docker:27017/?authSource=admin' `
>>   -e MONGO_DATABASE=inventario_curso `
>>   -e APP_VERSION=v1 `
>>   --network inventory_network `
>>   ttamayo08/inventario-api:1.0.0
8fac21f91be8af9e73ff06f96923eda32d9512d38ba21dab8c61b09dcd1f36f6
PS C:\Users\User\Projects\Proyecto_Docker>
```

Figura 27. *Creación de la API conectada a la misma red que MongoDB.*

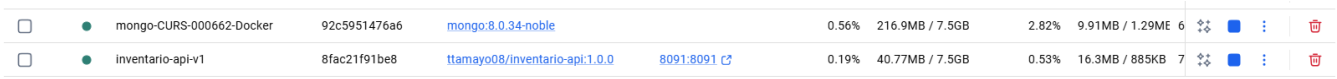


Figura 28. *Configuración de la conexión interna de la API usando el nombre del contenedor MongoDB.*

Se inspecciona la red y se confirma que los dos contenedores estan conectados a la misma red de inventory_network.

docker network inspect inventory_network



Figura 29. *Inspección de la red para comprobar los contenedores conectados.*

- Se crea un segundo contenedor del API.

```
docker run --name inventario-api-v2 -d `
 -p 8092:8091 `
 -e MONGO_URL='mongodb://root:123456@mongo-CURS-000662-
Docker:27017/?authSource=admin' `
 -e MONGO_DATABASE=inventario_curso `
 -e APP_VERSION=v2 `
 --network inventory_network `
 inventario-api:1.0.0
```

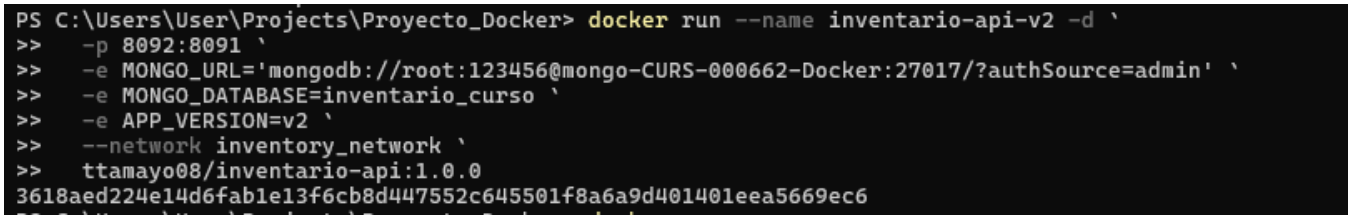


Figura 30. *Creación de una segunda instancia de la API con la imagen inventario-api 1.0.0.*

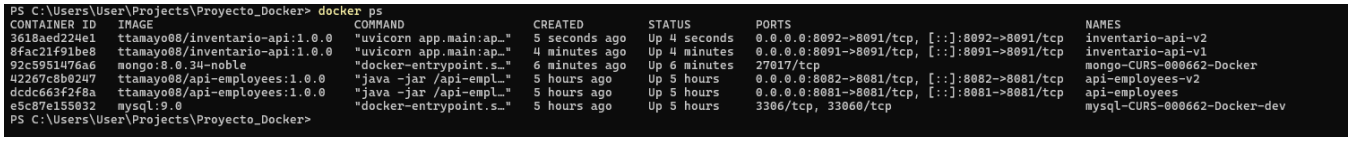


Figura 31. *Publicación de la segunda API en el puerto 8092.*

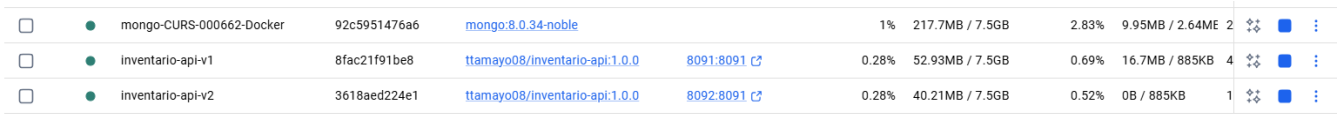


Figura 32. *Configuración de la versión v2 de la API.*

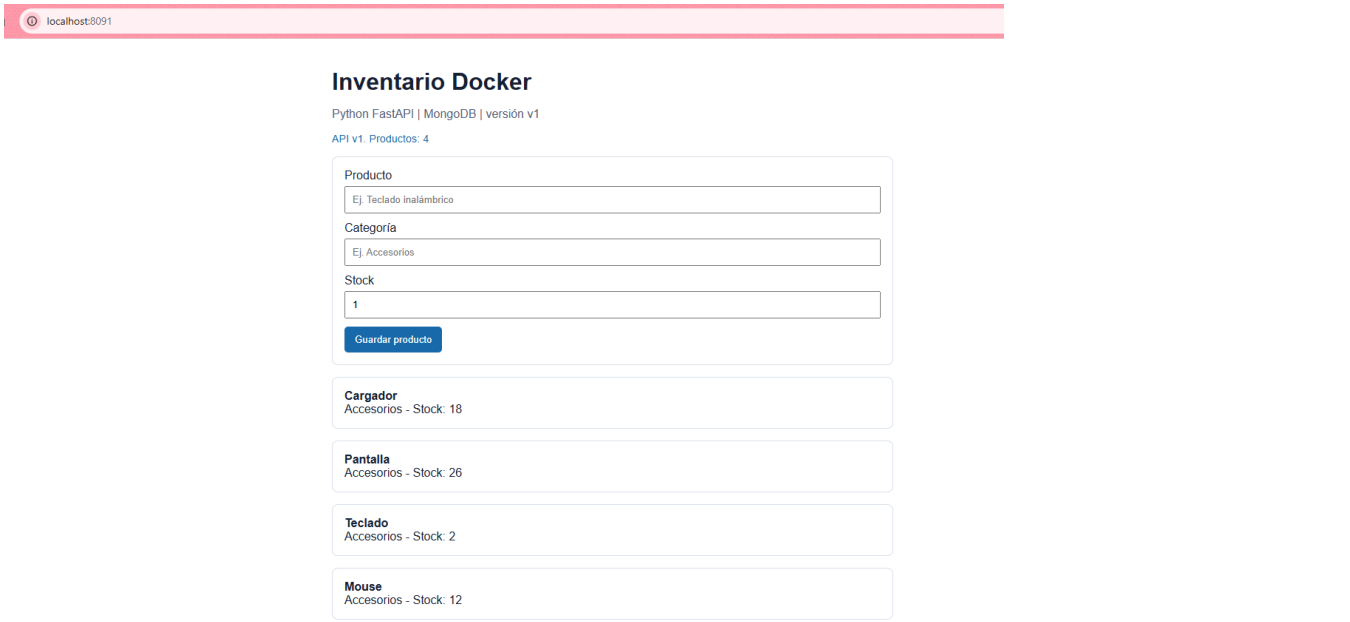


Figura 33. *Conexión interna de la segunda API con MongoDB.*

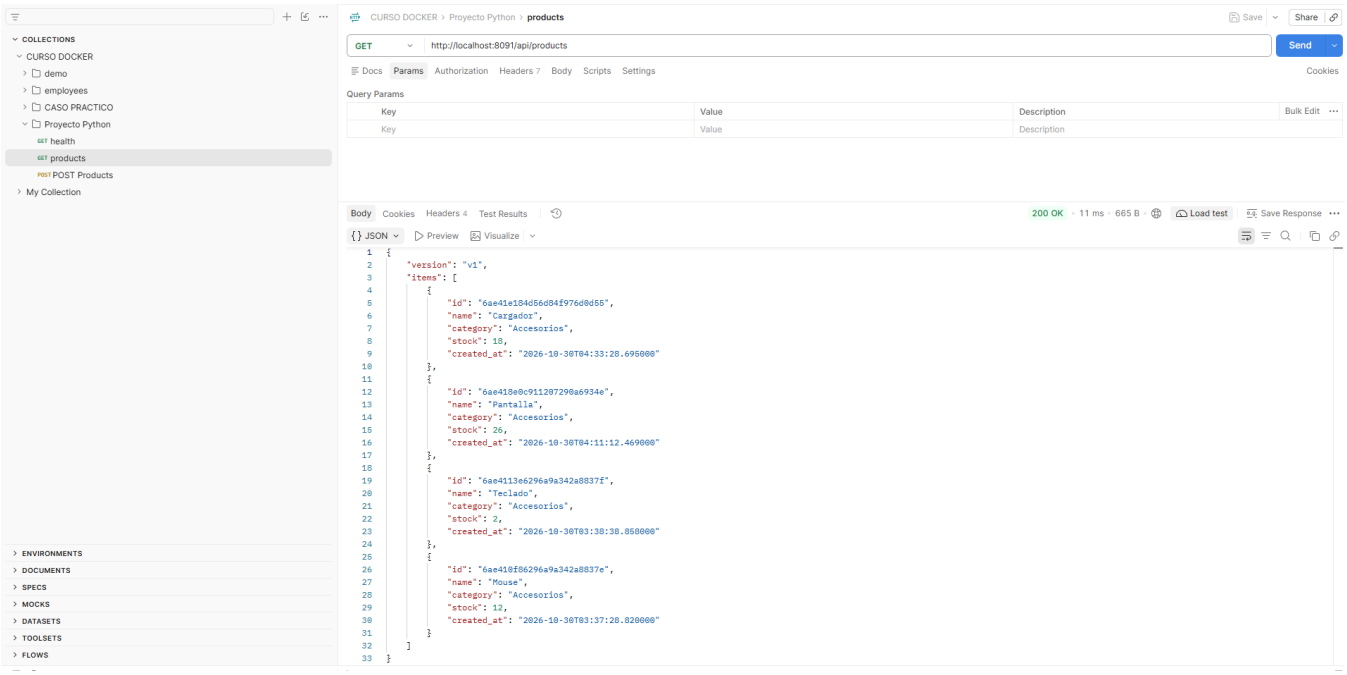


Figura 34. *Confirmación de ejecución del contenedor inventario-api-v2.*

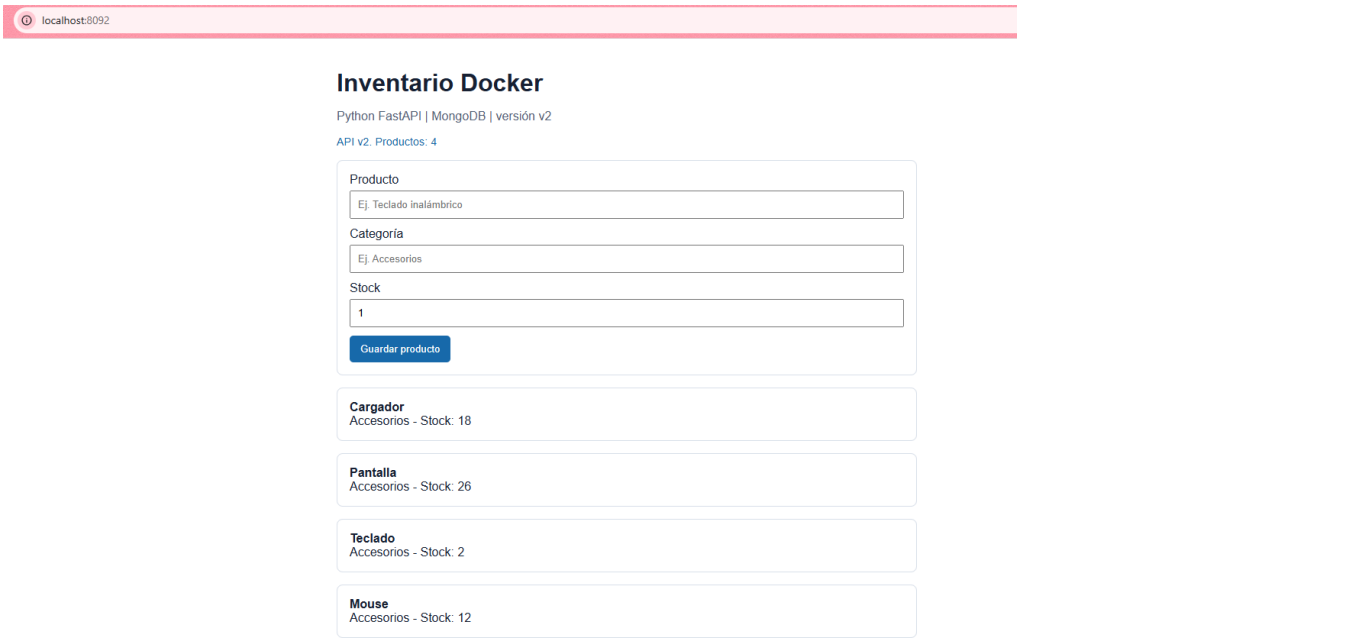


Figura 35. *Prueba del endpoint de la API versión 2.*

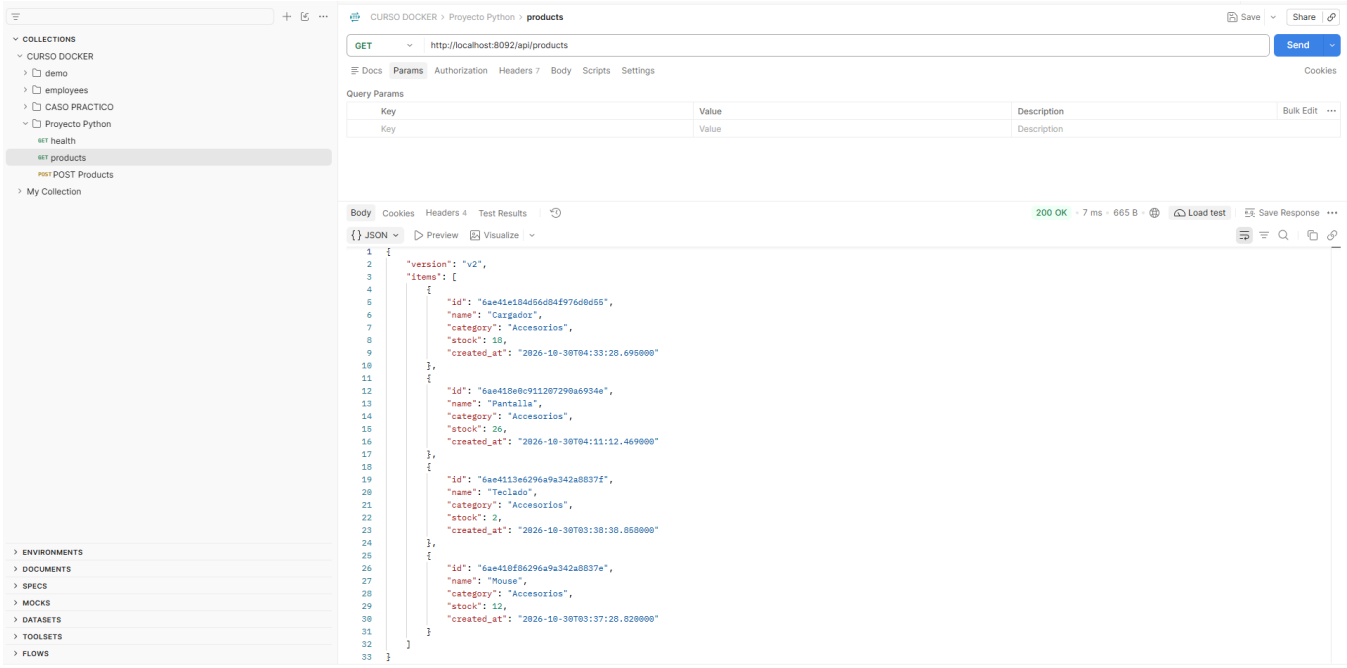


Figura 36. *Visualización de la interfaz correspondiente a la segunda instancia de la API.*

REGISTRO DE IMÁGENES DOCKER HUB

- Se sube imagen al repositorio de Docker Hub

```
PS C:\Users\User\Projects\Proyecto_Docker> docker login
Authenticating with existing credentials... [Username: ttamayo08]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login Succeeded
PS C:\Users\User\Projects\Proyecto_Docker> docker push ttamayo08/inventario-api:1.0.0
The push refers to repository [docker.io/ttamayo08/inventario-api]
0c83f555934c: Pushed
acca7542dfc7: Pushed
cf4a72b52b66: Pushed
3eb771151506: Pushed
653ede54d7c4: Pushed
f574edac59d9: Pushed
bede52ce8ab4: Pushed
6fc242d221a6: Pushed
1d9fde1df706: Pushed
1.0.0: digest: sha256:076243afc56db7fba72b803751fd8929837e3b590f0021dcf2b5f727519b7b7d size:
PS C:\Users\User\Projects\Proyecto_Docker> docker pull ttamayo08/inventario-api:1.0.0
1.0.0: Pulling from ttamayo08/inventario-api
Digest: sha256:076243afc56db7fba72b803751fd8929837e3b590f0021dcf2b5f727519b7b7d
Status: Image is up to date for ttamayo08/inventario-api:1.0.0
docker.io/ttamayo08/inventario-api:1.0.0
PS C:\Users\User\Projects\Proyecto_Docker>
```

Figura 37. *Inicio de sesión en Docker Hub para publicar la imagen creada.*

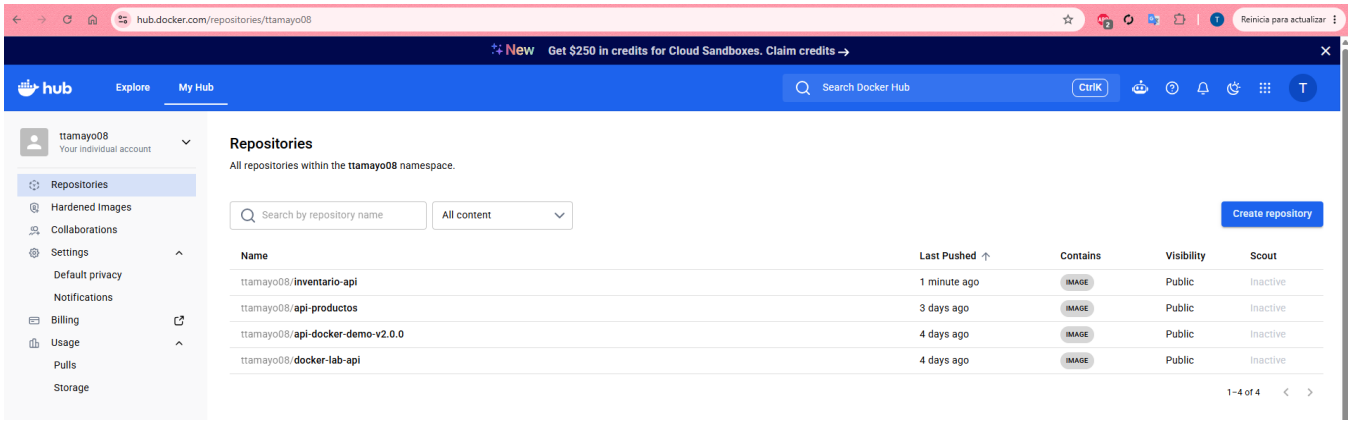


Figura 38. *Publicación y verificación del repositorio de la imagen en Docker Hub.*

DOCKER COMPOSE

Los cinco contenedores son mongo, redis, api-v1, api-v2 y gateway. La API v1 y la API v2 se crean desde el Dockerfile del proyecto; ambas usan la misma imagen propia, pero son contenedores independientes y se exponen en puertos distintos.

Configuración del archivo compose.yaml

C: > Users > User > Projects > Proyecto_Docker Compose > compose.yaml

```
1   name: inventory-compose-lab
2
3   services:
4   >  mongo: ···
23
24  >  redis: ···
38
39  >  api-v1: ···
71
72  >  api-v2: ···
104
105 >  gateway: ···
120
121 secrets:
122   mongo_root_password:
123     file: ./secrets/mongo_root_password.txt
124
125 volumes:
126   mongo_data:
127     name: compose-inventory-mongo-data
128   redis_data:
129     name: compose-inventory-redis-data
130
131 networks:
132   inventory_network:
133     name: compose-inventory-network
134
```

Figura 39. *Definición inicial del archivo compose.yaml para administrar el stack de servicios.*

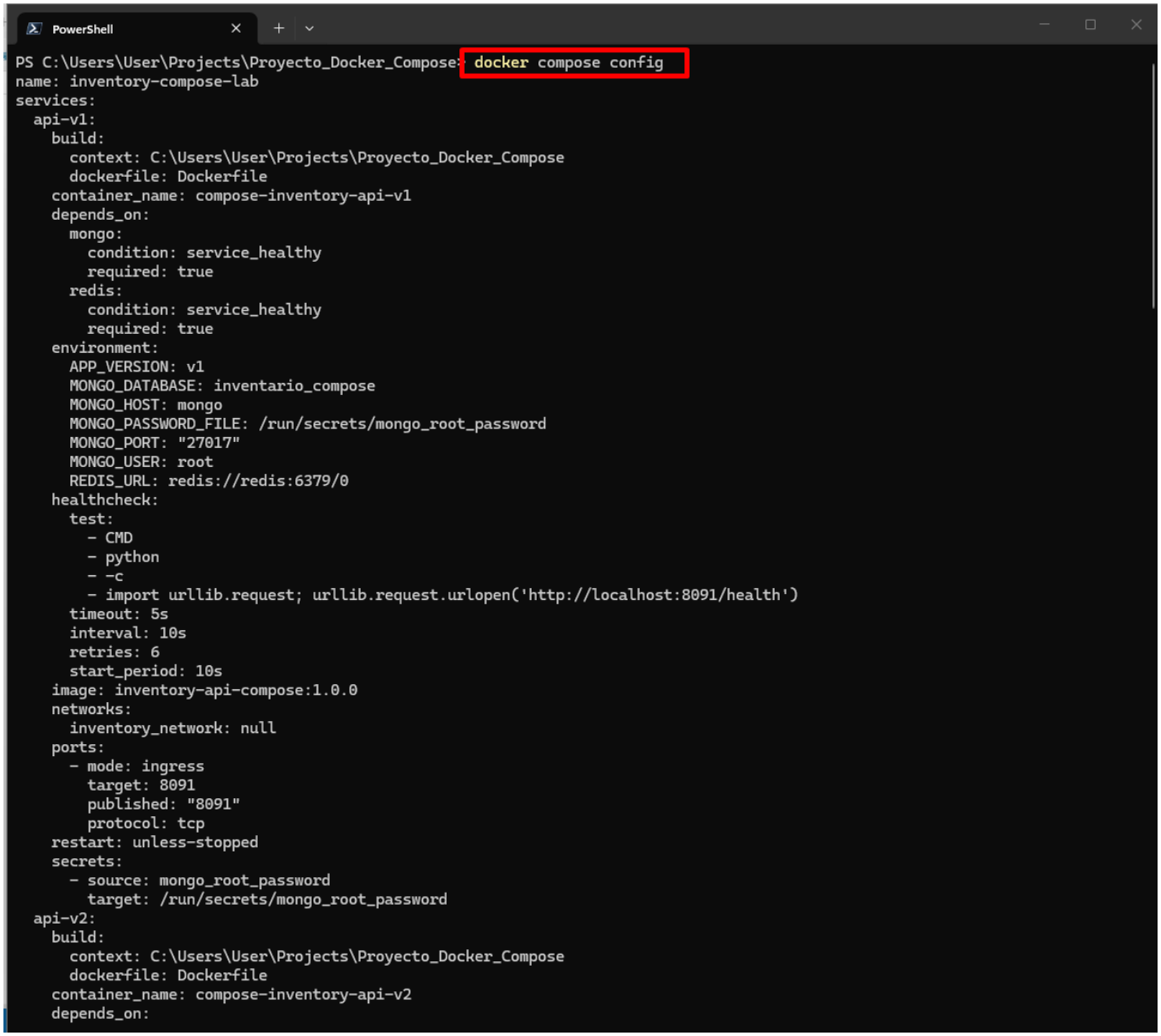


Figura 40. *Configuración del servicio MongoDB en Docker Compose.*

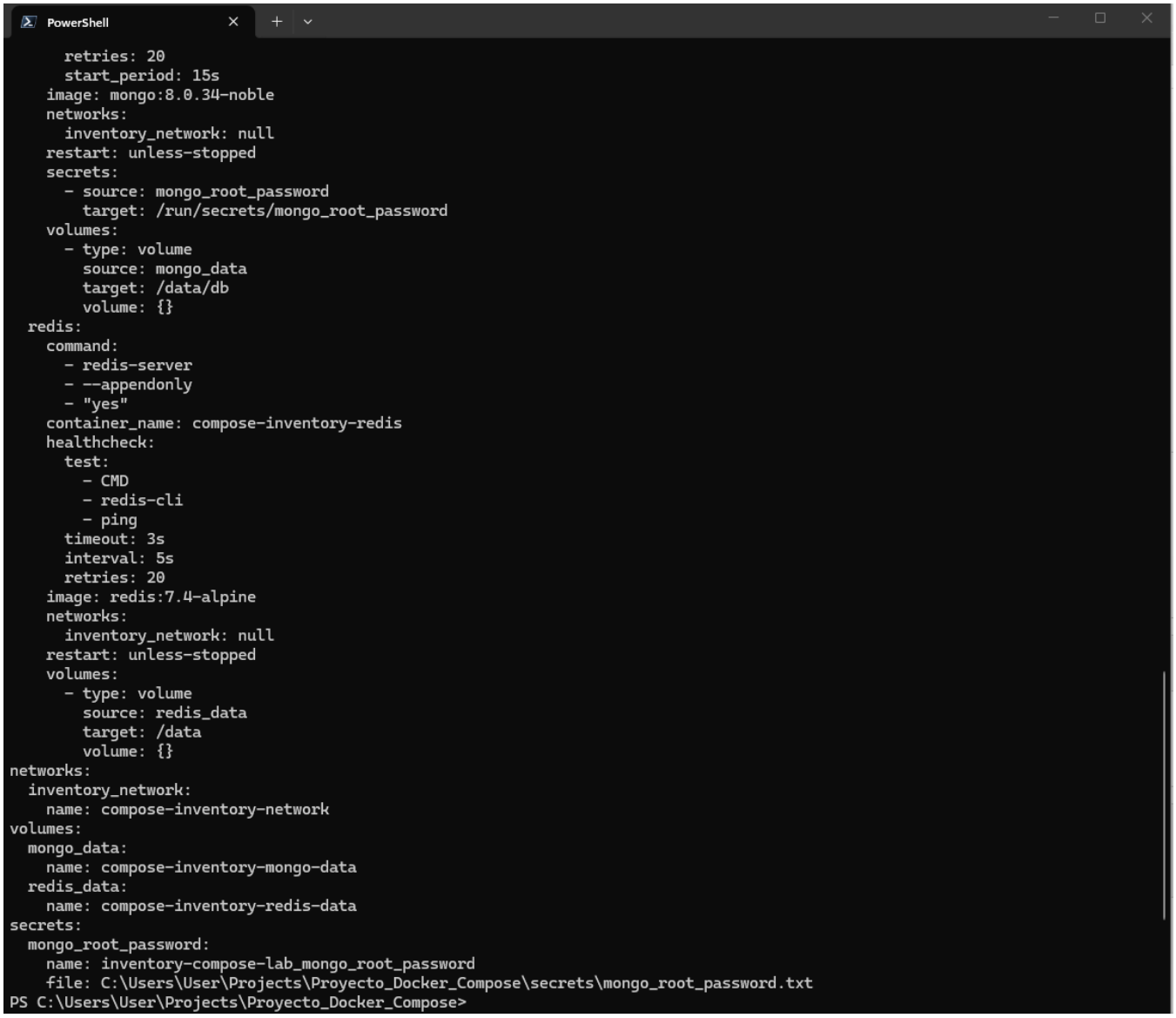


Figura 41. *Configuración del servicio Redis utilizado como servicio complementario.*

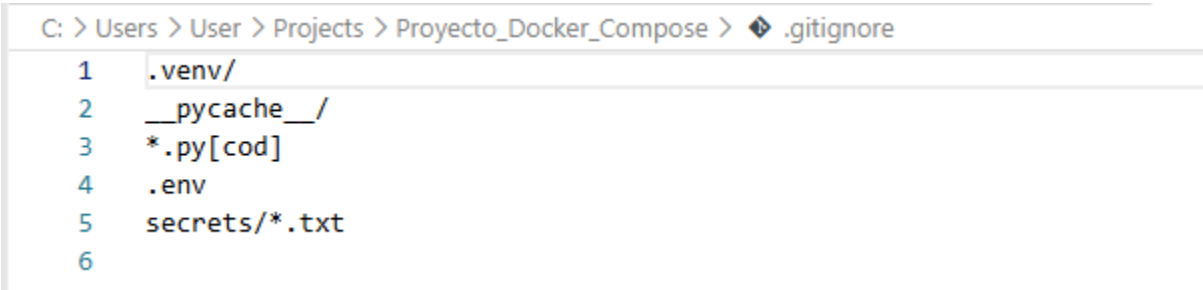


Figura 42. *Configuración del servicio api-v1, construido desde el Dockerfile.*

```
PS C:\Users\User> cd C:\Users\User\Projects\Proyecto_Docker_Compose
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose up --build -d
[+] Building 16.3s (14/14) FINISHED
 => [internal] load local bake definitions                                                                    0.0s
 => => reading from stdin 1.07kB                                                                              0.0s
 => [api-v2 internal] load build definition from Dockerfile                                                   0.0s
 => => transferring dockerfile: 348B                                                                          0.0s
 => [api-v1 internal] load metadata for docker.io/library/python:3.12-slim                                    0.0s
 => [api-v1 internal] load .dockerignore                                                                      0.0s
 => => transferring context: 91B                                                                              0.0s
 => [api-v2 1/5] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab  0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30  0.0s
 => [api-v2 internal] load build context                                                                      0.0s
 => => transferring context: 351B                                                                             0.0s
 => CACHED [api-v2 2/5] WORKDIR /app                                                                          0.0s
 => CACHED [api-v2 3/5] COPY requirements.txt .                                                               0.0s
 => [api-v1 4/5] RUN pip install --no-cache-dir -r requirements.txt                                          13.2s
 => [api-v2 5/5] COPY app ./app                                                                               0.1s
 => [api-v2] exporting to image                                                                               2.4s
 => => exporting layers                                                                                       1.8s
 => => exporting manifest sha256:accdacfa9fee19077ac54a28c1bec99899d0475f05aa387cf88247989ad76fde             0.0s
 => => exporting config sha256:7924371a12f8a771cfdf5475e0e2b93fdc02d5e3e60cecc7a44e2ef07ed4b25b               0.0s
 => => exporting attestation manifest sha256:2b07384d7a84265f9c74773cdae4ee02d81e1c033ef419be2cff986f97f800dd 0.0s
 => => exporting manifest list sha256:2c23dabbec347c21f6064e1fd4ccefd0acb3130c1a2be9503fa1ee6c4c98a553        0.0s
 => => naming to docker.io/library/inventory-api-compose:1.0.0                                                0.0s
 => => unpacking to docker.io/library/inventory-api-compose:1.0.0                                             0.5s
 => [api-v1] exporting to image                                                                               2.3s
 => => exporting layers                                                                                       1.8s
 => => exporting manifest sha256:90bb4f204672ac3e35eeb5c916c7bc9688fece7c0a756411856fd77b99570dbc             0.0s
 => => exporting config sha256:c5f8c0cdef714e507a796428e2ba2d5299c9124b4b1045b26e1e0d5c250e09af               0.0s
 => => exporting attestation manifest sha256:e4ec62678113b0bc7bf4f1fe3132dafcded0d43a2468799553d0d8fbeca93f2e 0.0s
 => => exporting manifest list sha256:aefc150c28933e029b666ffba30bab82b42dd12cc9462b26fded70d6ff57ae85        0.0s
 => => naming to docker.io/library/inventory-api-compose:1.0.0                                                0.0s
 => => unpacking to docker.io/library/inventory-api-compose:1.0.0                                             0.5s
 => [api-v1] resolving provenance for metadata file                                                           0.0s
 => [api-v2] resolving provenance for metadata file                                                           0.0s
[+] up 9/9
 ✔Image inventory-api-compose:1.0.0    Built                                                                16.5s
 ✔Network compose-inventory-network    Created                                                               0.0s
 ✔Volume compose-inventory-mongo-data  Created                                                               0.0s
 ✔Volume compose-inventory-redis-data  Created                                                               0.0s
 ✔Container compose-inventory-mongo    Healthy                                                               6.2s
 ✔Container compose-inventory-redis    Healthy                                                               5.7s
 ✔Container compose-inventory-api-v1   Healthy                                                              11.6s
 ✔Container compose-inventory-api-v2   Healthy                                                              12.1s
 ✔Container compose-inventory-gateway  Started                                                              12.3s
PS C:\Users\User\Projects\Proyecto_Docker_Compose>
```

Figura 43. *Configuración del servicio api-v2, publicado en un puerto diferente.*

```
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose ps
NAME                      IMAGE                          COMMAND                  SERVICE   CREATED              STATUS                        PORTS
compose-inventory-api-v1  inventory-api-compose:1.0.0    "uvicorn app.main:ap…"   api-v1    About a minute ago   Up About a minute (healthy)   0.0.0.0:8091->8091/tcp, [::]:8091->8091/tcp
compose-inventory-api-v2  inventory-api-compose:1.0.0    "uvicorn app.main:ap…"   api-v2    About a minute ago   Up About a minute (healthy)   0.0.0.0:8092->8091/tcp, [::]:8092->8091/tcp
compose-inventory-gateway nginx:1.27-alpine              "/docker-entrypoint.…"   gateway   About a minute ago   Up About a minute             0.0.0.0:8095->80/tcp, [::]:8095->80/tcp
compose-inventory-mongo   mongo:8.0.34-noble             "docker-entrypoint.s…"   mongo     About a minute ago   Up About a minute (healthy)   27017/tcp
compose-inventory-redis   redis:7.4-alpine               "docker-entrypoint.s…"   redis     About a minute ago   Up About a minute (healthy)   6379/tcp
PS C:\Users\User\Projects\Proyecto_Docker_Compose>
```

Figura 44. *Verificación de contenedores.*

```
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose images
CONTAINER                 REPOSITORY              TAG              PLATFORM       IMAGE ID       SIZE      CREATED
compose-inventory-api-v1  inventory-api-compose   1.0.0            linux/amd64    2c23dabbec34   267MB     2 minutes ago
compose-inventory-api-v2  inventory-api-compose   1.0.0            linux/amd64    2c23dabbec34   267MB     2 minutes ago
compose-inventory-gateway nginx                   1.27-alpine      linux/amd64    65645c7bb6a0   73.6MB    18 months ago
compose-inventory-mongo   mongo                   8.0.34-noble     linux/amd64    3c64937121b9   1.27GB    8 days ago
compose-inventory-redis   redis                   7.4-alpine       linux/amd64    8da19c7399dc   57.3MB    8 days ago
PS C:\Users\User\Projects\Proyecto_Docker_Compose>
```

Figura 45. *Declaración de la red, volúmenes y secreto usados por el stack Compose.*

- Verificación de cada servicio

Dirección	Servicio
http://localhost:8095	UI por Nginx Gateway hacia API v1
http://localhost:8091	UI servida directamente por API v1
http://localhost:8092	UI servida directamente por API v2
http://localhost:8095/v2/health	Verificación de API v2 mediante Nginx

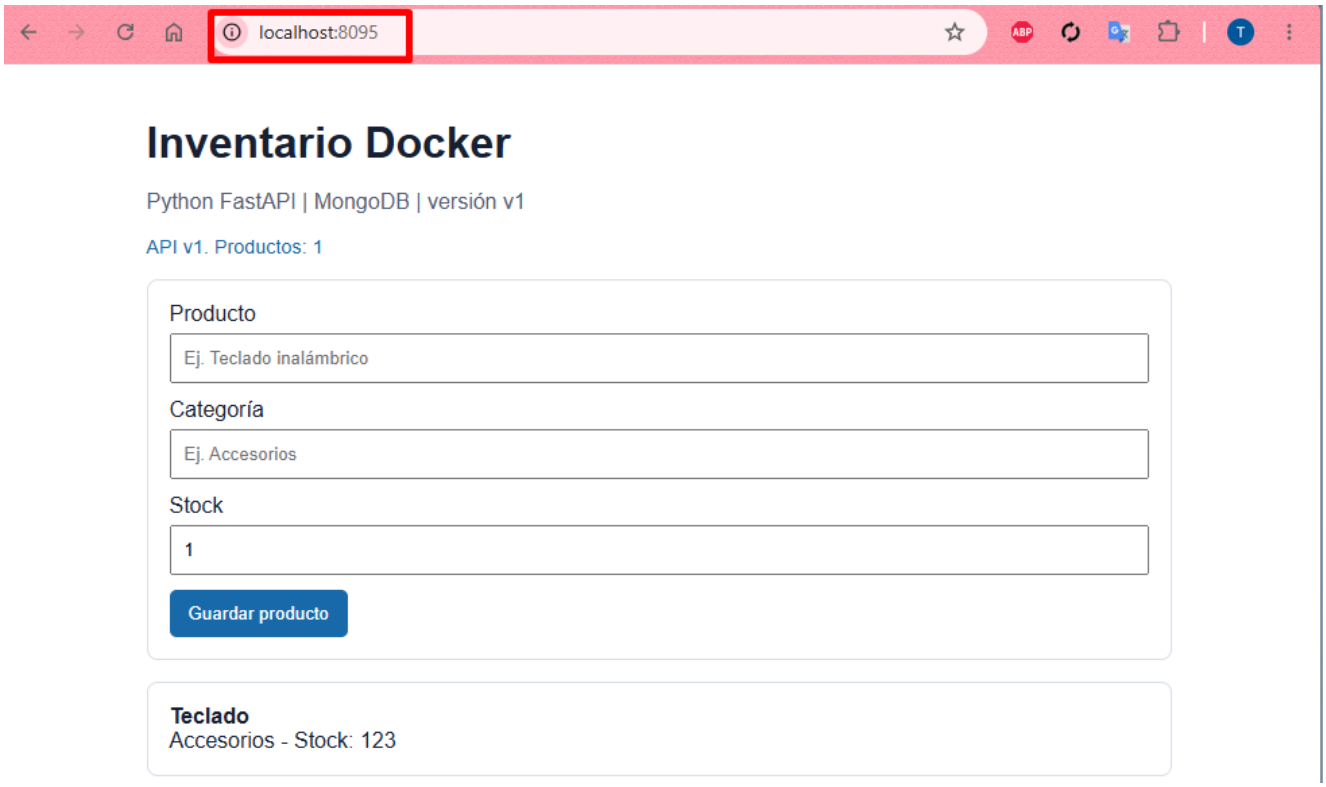


Figura 46. *Construcción e inicio de los servicios definidos en Docker Compose.*

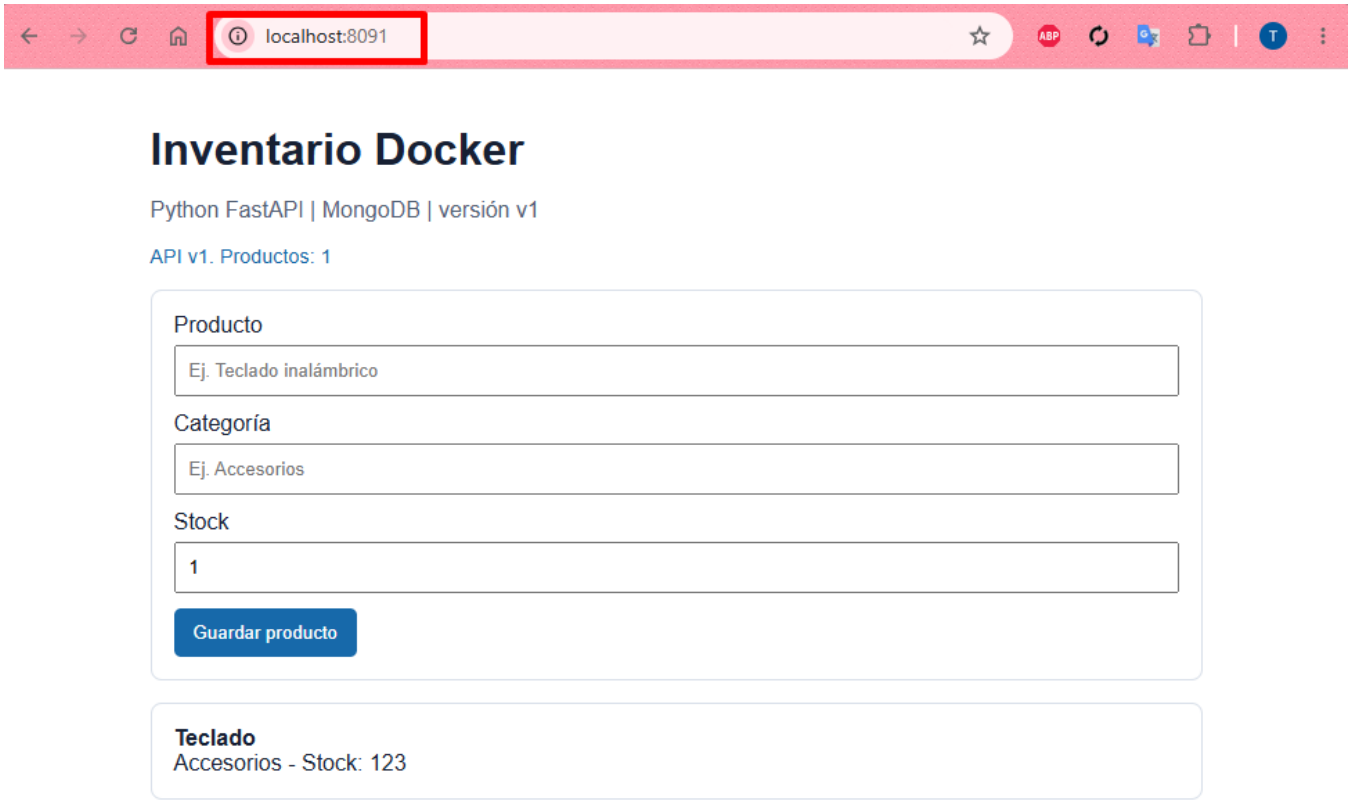


Figura 47. *Verificación de los cinco contenedores del proyecto mediante docker compose ps.*

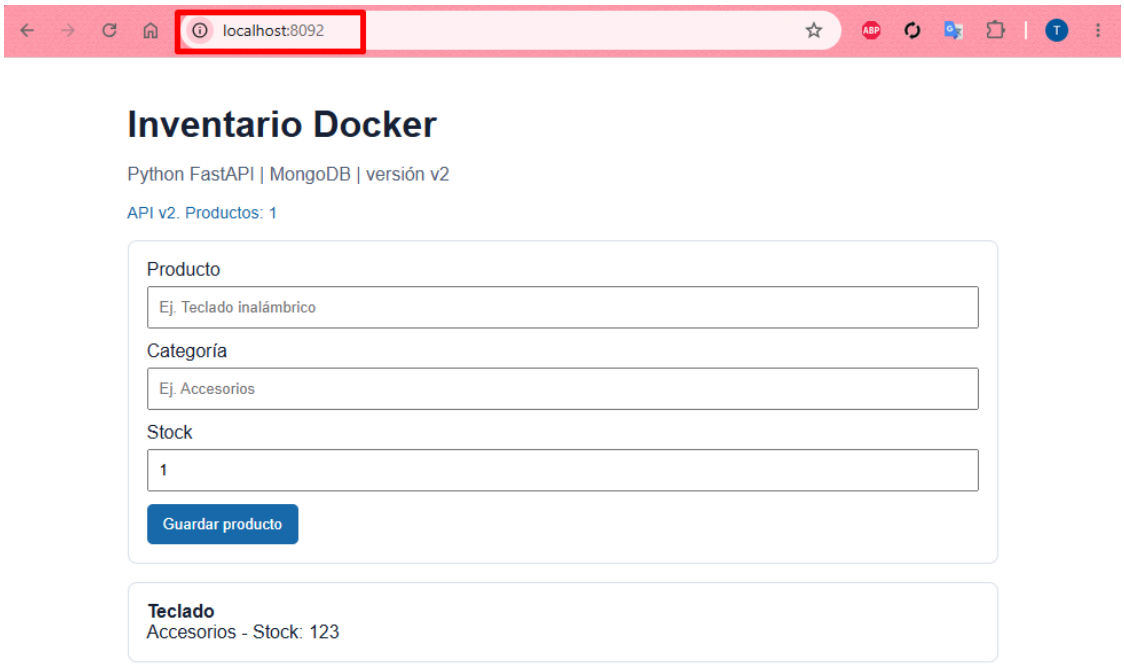


Figura 48. *Confirmación de puertos publicados y estado de los servicios.*

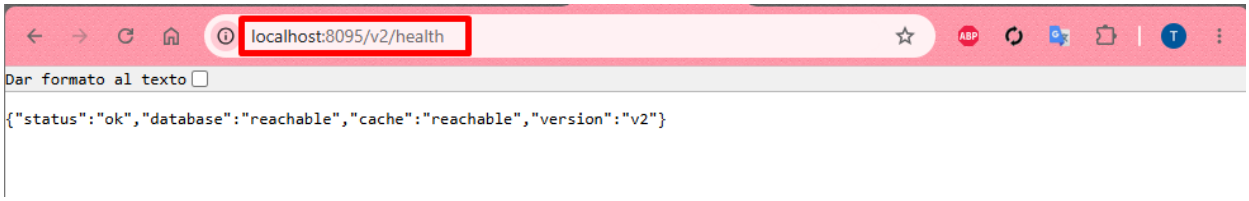


Figura 49. *Acceso a la interfaz por medio del gateway Nginx.*

Se comprueba API por Powershell

Invoke-RestMethod http://localhost:8091/health
Invoke-RestMethod http://localhost:8092/health
Invoke-RestMethod -Method Post -Uri http://localhost:8091/api/products -ContentType 'application/json' -Body '{"name":"Teclado","category":"Accesorios","stock":8}'
Invoke-RestMethod http://localhost:8092/api/products

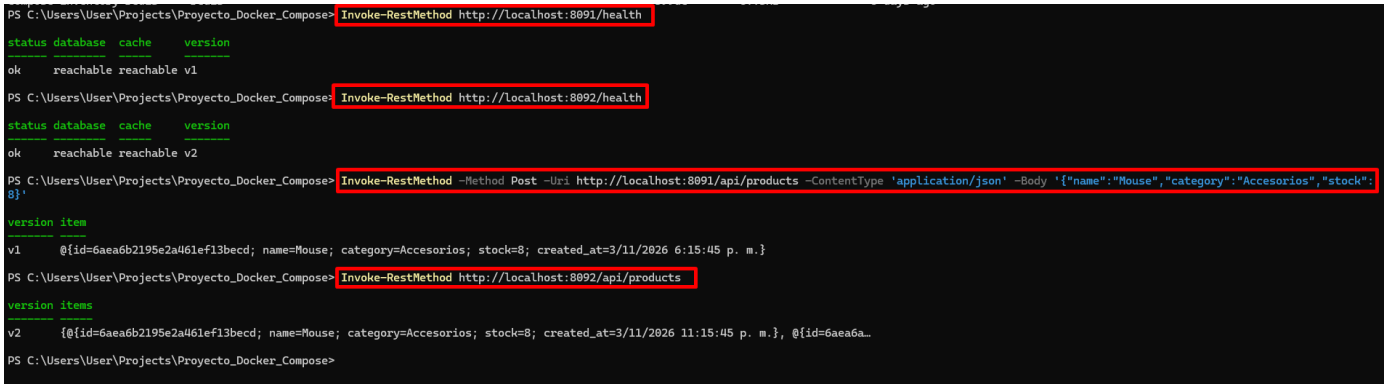


Figura 50. *Consulta de productos a través de la API versión 2.*

Se inspecciona la red creada y se verifica los servicios.

```
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker network inspect compose-inventory-network
[
    {
        "Name": "compose-inventory-network",
        "Id": "5bda10f0d93d88199f8beddc3d618b056b16b6d097fa7cf804bf03c85a413557",
        "Created": "2026-11-03T23:06:24.9788627747",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.20.0.0/16",
                    "Gateway": "172.20.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {
            "com.docker.compose.config-hash": "04651d9900d5a58a8b3d03515b56ef3af28af8a6615c783e4594fdbae47763b3",
            "com.docker.compose.network": "inventory_network",
            "com.docker.compose.project": "inventory-compose-lab",
            "com.docker.compose.version": "5.6.1"
        },
        "Containers": {
            "2431960cce145a1687d065f1e07efefafc897f208d561353ec4c564fd8057a4f": {
                "Name": "compose-inventory-redis",
                "EndpointID": "9d996289ccfcf028c7968535fd66e2cba1521d5eacf0ec5c057f0e2d3e0a9247",
                "MacAddress": "66:49:e3:74:3b:f5",
                "IPv4Address": "172.20.0.2/16",
                "IPv6Address": ""
            },
            "24f17eeaa070084e68f5d3fe831884f76213f2e12b88da39bd0d51211fcc56cc": {
                "Name": "compose-inventory-mongo",
                "EndpointID": "7accb6c912a0919f90fd1303944a74b3eacd5c81a46b1ee8f98f7287354990ab",
                "MacAddress": "de:ea:7a:bd:d7:50",
                "IPv4Address": "172.20.0.3/16",
                "IPv6Address": ""
            },
            "70b9be14c154bcb0e1943406d6dd4ddbdf8989e6fcb9c7b395e79045c48201b4": {
                "Name": "compose-inventory-api-v1",
                "EndpointID": "4d5413bf8b755e3e029848d8fe83bd88e3ff3c6a6f243b286ce770f4cc6123f4",
                "MacAddress": "56:88:4a:de:2f:e4",
```

Figura 51. Inspección de la red creada automáticamente por Docker Compose.

```
            },
            "7c29e8697edae36877f5195b73a834d2884c2a38c96739bf647495cb0bd76a5e": {
                "Name": "compose-inventory-api-v2",
                "EndpointID": "9e36006b0667d02e77e278d286bfc42a7fdd37c7d325c444e83aade042f233ea",
                "MacAddress": "b6:77:fd:9b:73:09",
                "IPv4Address": "172.20.0.5/16",
                "IPv6Address": ""
            },
            "d8b0a1490e979486cf48104ad844237a6ef95cb668b6e098f2a0b35af91bc461": {
                "Name": "compose-inventory-gateway",
                "EndpointID": "b29027a8f39b2dcf23aa9d43367f723a8b6d0df8b4ee8f717261986cd3b66fb9",
                "MacAddress": "ba:dd:86:b0:25:76",
                "IPv4Address": "172.20.0.6/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.20.0.0/16": {
                        "IPsInUse": 8,
                        "DynamicIPsAvailable": 65528
                    }
                }
            }
        }
    }
]
PS C:\Users\User\Projects\Proyecto_Docker_Compose>
```

Figura 52. Confirmación de que los servicios del stack comparten la red interna.

- Se comprueba desde dentro del contenedor api-v1 que el secreto de Docker fue montado correctamente como archivo.

docker compose exec api-v1 sh -c "test -f /run/secrets/mongo_root_password && echo Secret-montado-correctamente"



Figura 53. *Comprobación de que el secreto de MongoDB fue montado correctamente en api-v1.*

Se ingresa al contenedor de DB y se consulta la DB de productos

docker compose exec mongo mongosh -u root -p 123456 --authenticationDatabase admin

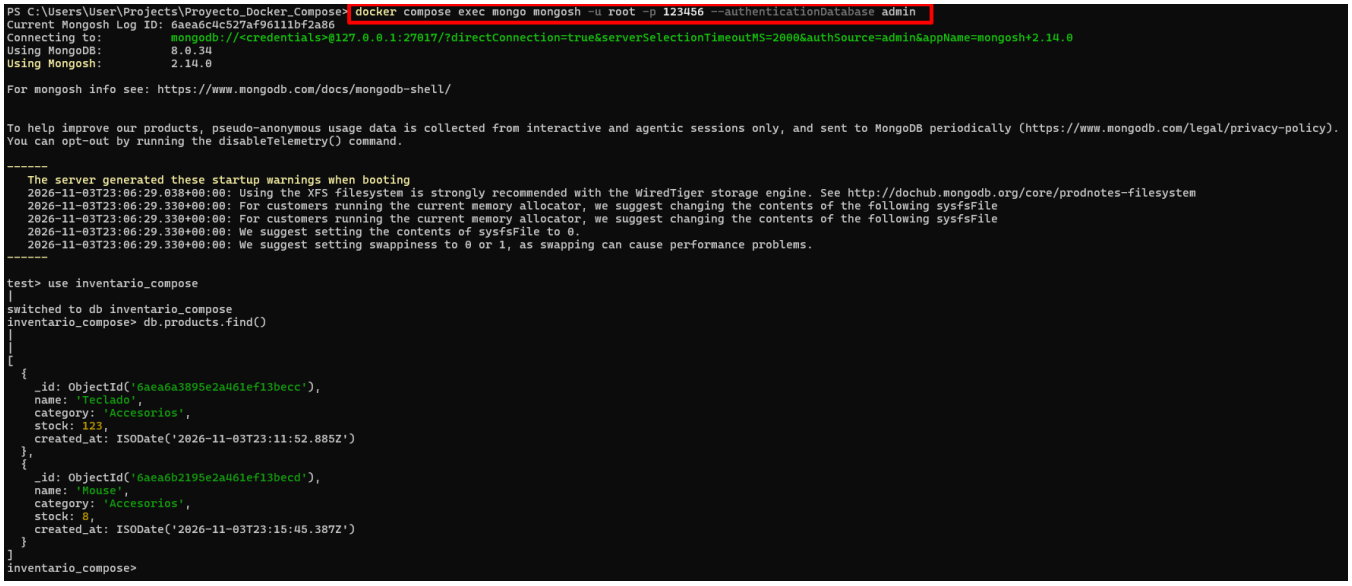


Figura 54. *Acceso a MongoDB desde Compose y consulta de los productos almacenados.*

DOCKER COMPOSE CON BUILD

Se realizará el cambio del titulo del aplicativo en caliente

C: > Users > User > Projects > Proyecto_Docker_Compose > app > templates > index.html > html > body

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Inventario Docker Compose Tatiana </title>
    <style>
      body { font-family: Arial, sans-serif; max-width: 760px; margin: 40px auto; padding: 0 18px; color: #172033; }
      h1 { margin-bottom: 4px; }
      p { color: #526075; }
      form, li { border: 1px solid #d9e0ea; border-radius: 8px; padding: 16px; margin-top: 16px; }
      label, input { display: block; width: 100%; box-sizing: border-box; }
      input { padding: 9px; margin: 5px 0 11px; }
      button { background: #1769aa; color: white; border: 0; border-radius: 5px; padding: 10px 14px; cursor: pointer; }
      ul { padding: 0; list-style: none; }
      .status { font-size: 0.9rem; color: #1769aa; }
    </style>
  </head>
  <body>
    <h1>Inventario Docker Compose Tatiana </h1>
    <p>Python FastAPI | MongoDB | versión {{ version }}</p>
    <p id="status" class="status">Conectando con la API...</p>

```

Figura 55. *Modificación del título de la interfaz para demostrar un cambio en el código fuente.*

- Se reconstruye solo las APIs

docker compose up --build -d api-v1 api-v2

```
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose up --build -d api-v1 api-v2
[+] Building 1.0s (14/14) FINISHED
 => [internal] load local bake definitions                                                                                                                0.0s
 => => reading from stdin 1.07kB                                                                                                                          0.0s
 => [api-v2 internal] load build definition from Dockerfile                                                                                               0.0s
 => => transferring dockerfile: 308B                                                                                                                      0.0s
 => [api-v1 internal] load metadata for docker.io/library/python:3.12-slim                                                                                0.1s
 => [api-v1 internal] load .dockerignore                                                                                                                  0.0s
 => => transferring context: 91B                                                                                                                          0.0s
 => [api-v2 1/5] FROM docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30                          0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:b18b261e1b4974b4f321451ad5251ded85abb22c2b8b262da527dda1d031ab30                                  0.0s
 => [api-v2 internal] load build context                                                                                                                  0.0s
 => => transferring context: 3.23kB                                                                                                                       0.0s
 => CACHED [api-v1 2/5] WORKDIR /app                                                                                                                      0.0s
 => CACHED [api-v1 3/5] COPY requirements.txt .                                                                                                           0.0s
 => CACHED [api-v1 4/5] RUN pip install --no-cache-dir -r requirements.txt                                                                                0.0s
 => [api-v2 5/5] COPY app ./app                                                                                                                           0.0s
 => [api-v2] exporting to image                                                                                                                           0.2s
 => => exporting layers                                                                                                                                   0.1s
 => => exporting manifest sha256:dba865364f9ffaa066724e712f1b34ae2486eec98e2ddb8cddf03a5138186534f                                                         0.0s
 => => exporting config sha256:9c94750f76b5969dc7937846a1b682b57a3cca809b61095a82be3fb39b31eac6                                                           0.0s
 => => exporting attestation manifest sha256:aed6e9e1da71ce4855247686a1d3daf6cb8e30a5815da78c2f62cc627ffd9e01                                             0.0s
 => => exporting manifest list sha256:7a24497d01b0b36bf7989afece96927e9d59c4d861d2951a35459a1bb5da706c                                                    0.0s
 => => naming to docker.io/library/inventory-api-compose:1.0.0                                                                                            0.0s
 => => unpacking to docker.io/library/inventory-api-compose:1.0.0                                                                                         0.0s
 => [api-v1] exporting to image                                                                                                                           0.2s
 => => exporting layers                                                                                                                                   0.1s
 => => exporting manifest sha256:da191b691d854d027676c26ea57855b95870da9f16dfdcfa7499f5086bdb366d                                                         0.0s
 => => exporting config sha256:7359b9d5959f0e20a31b8e61f497164c62a36dc1b80ca7777747ea9a10d98932                                                           0.0s
 => => exporting attestation manifest sha256:c8edb3e248b9bcd004b3650ae35d4a409a051e41af5d67c159ab5a2fb23d386d                                             0.0s
 => => exporting manifest list sha256:3eea8f1bbfc7e06fe2a3f9a89e037dd7be45947fac928d1507986fcecdab4a65                                                    0.0s
 => => naming to docker.io/library/inventory-api-compose:1.0.0                                                                                            0.0s
 => => unpacking to docker.io/library/inventory-api-compose:1.0.0                                                                                         0.0s
 => [api-v1] resolving provenance for metadata file                                                                                                       0.0s
 => [api-v2] resolving provenance for metadata file                                                                                                       0.0s
[+] up 5/5
 ✔ Image inventory-api-compose:1.0.0  Built                                                                                                               1.2s
 ✔ Container compose-inventory-mongo  Healthy                                                                                                             1.8s
 ✔ Container compose-inventory-redis  Healthy                                                                                                             1.8s
 ✔ Container compose-inventory-api-v1 Started                                                                                                             2.1s
 ✔ Container compose-inventory-api-v2 Started                                                                                                             2.3s
PS C:\Users\User\Projects\Proyecto_Docker_Compose>
```

Figura 56. *Reconstrucción de api-v1 y api-v2 después de modificar el código.*

```
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose ps
NAME                      IMAGE                          COMMAND                  SERVICE     CREATED          STATUS                    PORTS
compose-inventory-api-v1    inventory-api-compose:1.0.0    "uvicorn app.main:ap…"   api-v1      47 seconds ago   Up 45 seconds (healthy)   0.0.0.0:8091->8091/tcp, [::]:8091->8091/tcp
compose-inventory-api-v2    inventory-api-compose:1.0.0    "uvicorn app.main:ap…"   api-v2      47 seconds ago   Up 45 seconds (healthy)   0.0.0.0:8092->8091/tcp, [::]:8092->8091/tcp
compose-inventory-gateway   nginx:1.27-alpine              "/docker-entrypoint.…"   gateway     27 minutes ago   Up 27 minutes             0.0.0.0:8095->80/tcp, [::]:8095->80/tcp
compose-inventory-mongo     mongo:8.0.34-noble             "docker-entrypoint.s…"   mongo       27 minutes ago   Up 27 minutes (healthy)   27017/tcp
compose-inventory-redis     redis:7.4-alpine               "docker-entrypoint.s…"   redis       27 minutes ago   Up 27 minutes (healthy)   6379/tcp
PS C:\Users\User\Projects\Proyecto_Docker_Compose>
```

Figura 57. *Resultado de la construcción de las imágenes después del cambio realizado.*

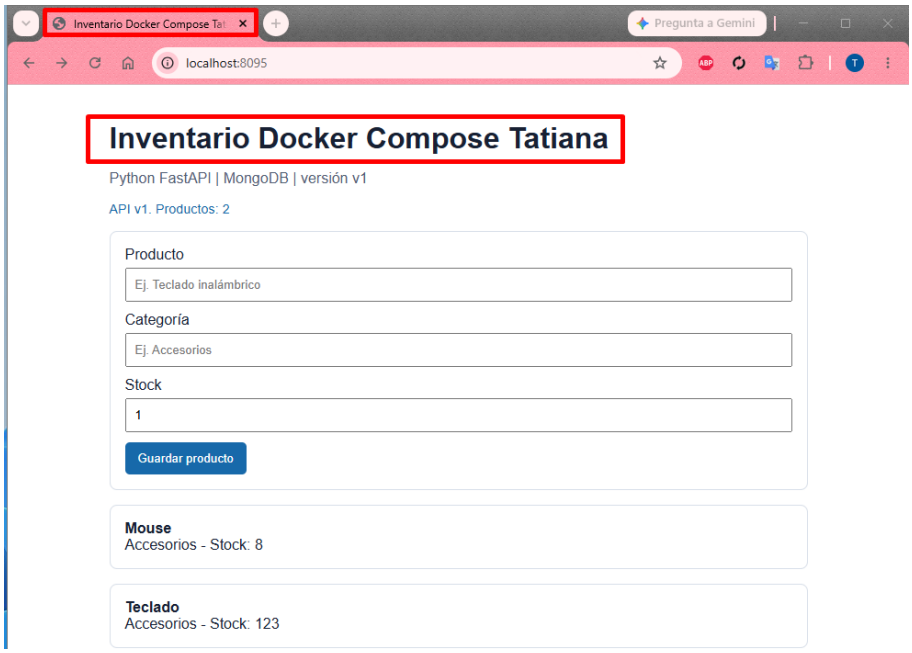


Figura 58. *Verificación de que los servicios reconstruidos están nuevamente activos.*

SECRETS Y VOLUMEN PERSISTENTE

- Demostracion persistencia de datos

```
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose down
[+] down 6/6
 ✔Container compose-inventory-gateway Removed                    0.4s
 ✔Container compose-inventory-api-v1  Removed                    0.9s
 ✔Container compose-inventory-api-v2  Removed                    0.5s
 ✔Container compose-inventory-redis   Removed                    0.3s
 ✔Container compose-inventory-mongo   Removed                    0.4s
 ✔Network compose-inventory-network   Removed                    0.3s

PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker volume ls
DRIVER    VOLUME NAME
local     4ed0c3ab154f5d46cae8c7a2fa81fa20a9dacd101d02e76aa689cc4b06ab109f
local     5c24c760095866f08102067b6f3074f6e71f8974f5ba2f9bcdb82a54a1180c96
local     76dd46dfcb8375a6a37b0bfcfaebba3e65c1b50cd8a6ab27b8dc9c6d17cbc812
local     b45732aaf8c4bfe4c009a9d25d2754791947f0e7c1e16b6ec706fa727d749e34
local     cfe61e0a7852d94bcc2c2d481a5c68a0baa319cc6a6e4ccd8f79c65b7ccc18c5
local     compose-inventory-mongo-data
local     compose-inventory-redis-data
local     mongo-CURS-000662-data
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose up -d
[+] up 6/6
 ✔Network compose-inventory-network   Created                    0.0s
 ✔Container compose-inventory-mongo   Healthy                    6.2s
 ✔Container compose-inventory-redis   Healthy                    6.2s
 ✔Container compose-inventory-api-v2  Healthy                   11.5s
 ✔Container compose-inventory-api-v1  Healthy                   12.0s
 ✔Container compose-inventory-gateway Started                   12.2s
```

Figura 59. *Inicio del stack después de detenerlo como parte de la prueba de persistencia.*

OTROS COMANDO DE INVESTIGACIÓN

- Ver el consumo de CPU, memoria y red de los contenedores

```
PS C:\Users\User> docker stats
```

Figura 60. *Consulta del consumo de CPU, memoria y red de los contenedores.*

```
CONTAINER ID   NAME                          CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O         PIDS
042438a652d2   compose-inventory-gateway     0.00%     10.93MiB / 7.505GiB   0.14%     26.4kB / 25.4kB   0B / 4.1kB        15
1ae3c97905b4   compose-inventory-api-v1      0.17%     43.46MiB / 7.505GiB   0.57%     139kB / 105kB     0B / 893kB        9
32d142b90793   compose-inventory-api-v2      0.19%     43.33MiB / 7.505GiB   0.56%     127kB / 85.2kB    0B / 893kB        9
55b3f3b1e723   compose-inventory-redis       0.46%     3.504MiB / 7.505GiB   0.05%     42.5kB / 20.4kB   139kB / 0B        6
b8e7634199b3   compose-inventory-mongo       0.54%     204.6MiB / 7.505GiB   2.66%     135kB / 233kB     0B / 9.18MB       54
```

Figura 61. *Monitoreo de recursos utilizados por los servicios Docker.*

- Ver los eventos recientes de Docker: creación, inicio, conexiones, etc.

docker events --since 10m



Figura 62. *Visualización de eventos recientes generados por Docker.*

- Consultar el tamaño ocupado por imágenes, contenedores, caché y volúmenes

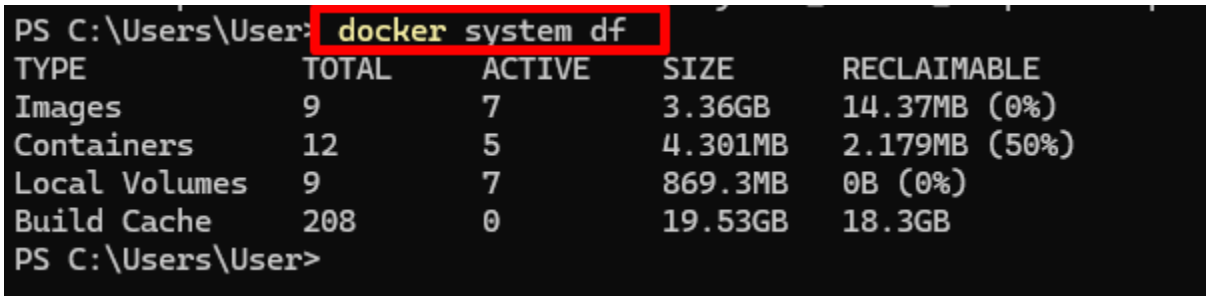


Figura 63. *Consulta del espacio ocupado por imágenes, contenedores, caché y volúmenes.*

- Ver las capas e instrucciones con las que se creó una image

docker image history inventory-api-compose:1.0.0

```
PS C:\Users\User> docker image history inventory-api-compose:1.0.0
IMAGE          CREATED             CREATED BY                                      SIZE      COMMENT
d29ff599714e   28 minutes ago      CMD ["uvicorn" "app.main:app" "--host" "0.0.…   0B        buildkit.dockerfile.v0
<missing>      28 minutes ago      EXPOSE [8091/tcp]                               0B        buildkit.dockerfile.v0
<missing>      28 minutes ago      COPY app ./app # buildkit                       41kB      buildkit.dockerfile.v0
<missing>      59 minutes ago      RUN /bin/sh -c pip install --no-cache-dir -r…   69.6MB    buildkit.dockerfile.v0
<missing>      About an hour ago   COPY requirements.txt . # buildkit              12.3kB    buildkit.dockerfile.v0
<missing>      About an hour ago   WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      About an hour ago   LABEL version=1.0.0                             0B        buildkit.dockerfile.v0
<missing>      About an hour ago   LABEL name=API de gestión de inventario         0B        buildkit.dockerfile.v0
<missing>      About an hour ago   LABEL build_date=29-10-2026                     0B        buildkit.dockerfile.v0
<missing>      6 days ago          CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      6 days ago          RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      6 days ago          RUN /bin/sh -c set -eux;   savedAptMark="$(a…   41.4MB    buildkit.dockerfile.v0
<missing>      6 days ago          ENV PYTHON_SHA256=b5796915dc45ff745cfb4e990a…   0B        buildkit.dockerfile.v0
<missing>      6 days ago          ENV PYTHON_VERSION=3.12.16                      0B        buildkit.dockerfile.v0
<missing>      6 days ago          ENV GPG_KEY=7169605F62C751356D054A26A821E680…   0B        buildkit.dockerfile.v0
<missing>      6 days ago          RUN /bin/sh -c set -eux;  apt-get update;  a…   4.93MB    buildkit.dockerfile.v0
<missing>      6 days ago          ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      6 days ago          ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      8 days ago          # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.7MB    debuerreotype 0.17
PS C:\Users\User>
```

Figura 64. *Visualización de las capas que componen la imagen inventory-api-compose 1.0.0.*

- Consultar las variables de entorno disponibles dentro de la API

docker compose exec api-v1 printenv

```
PS C:\Users\User\Projects\Proyecto_Docker_Compose> docker compose exec api-v1 printenv
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=1ae3c97905b4
TERM=xterm
MONGO_HOST=mongo
MONGO_PORT=27017
MONGO_USER=root
MONGO_PASSWORD_FILE=/run/secrets/mongo_root_password
MONGO_DATABASE=inventario_compose
REDIS_URL=redis://redis:6379/0
APP_VERSION=v1
LANG=C.UTF-8
GPG_KEY=7169605F62C751356D054A26A821E680E5FA6305
PYTHON_VERSION=3.12.16
PYTHON_SHA256=b5796915dc45ff745cfb4e990a7e7c93d6f546328a102ce31448d69ebc37575c
HOME=/root
```

Figura 65. *Consulta de las variables de entorno disponibles dentro del contenedor api-v1.*